

Certified Stall Recovery Procedures Measured Against the Global Optimum: A Dynamic Programming Approach

Nicolás D. Romero*

Business or Academic Affiliation 1, City, State, Zip Code, Country

Gabriel G. Torre†

Business or Academic Affiliation 2, City, State, Zip Code, Country

Roberto A. Bunge‡

Business or Academic Affiliation 4, City, State, Zip Code, Country

Juan I. Giribet§

Business or Academic Affiliation 3, City, Province, Zip Code, Country

Loss of control in flight, most often initiated by a stall, remains the leading cause of fatal accidents in general aviation. A stall recovery is a two-phase maneuver: a nose-down input to restore attached flow, then a pullout that arrests the descent. It is measured by the altitude it costs, and the certified procedures do not agree on how to fly it: one commands full power and nose-down together, the other delays power until the wing is flying again. Cast as an undiscounted infinite-horizon problem, the recovery admits a globally optimal solution by dynamic programming (DP), optimal over the whole entry envelope rather than at a nominal condition, and that guarantee is what the method is built to keep. Discretizing the state space forces the value function to be interpolated; because barycentric interpolation is non-expansive [1], the backup retains the convergence of the exact recursion and delivers the exact optimum of the discretized problem: the policies reported here carry a guarantee, not a heuristic's promise. Integrating the flight dynamics inside the GPU kernel lets that grid be fine enough to resolve the recovery. On the 3-DOF banked pullout of Bunge et al. [2] the solver resolves 31.9 million states and 91 actions in 15 minutes on a single GPU and agrees with a direct-collocation benchmark to within 1.4% of the maximum altitude lost. Applied to a realistic recovery problem, a 4-DOF symmetric stall over 14.9 million states and 451 actions with aerodynamic coefficients measured in the wind tunnel through stall and post-stall, their coupling to propulsion, and the first-order engine lag of Riley [3], the optimal policy reproduces the shape of the simultaneous-power procedure: full throttle across essentially the entire recovery envelope, an elevator switching surface pinned to the stall boundary rather than to a clock, and a pull that holds the angle of attack within a degree of that boundary. Across 1000 initial conditions of the stalled-entry set the ordering optimum, then simultaneous power, then unstall-gated power, then power-delayed holds at every entry, consistent with the flight-test ranking of Gratton et al. [4], and the exact solution decomposes the margin: most of what either certified sequence concedes is the power ramp rather than the doctrinal difference between them. Across $\pm 15\%$ weight and the achievable CG range the nominal policy continues to recover, and re-solving it for the perturbed aircraft buys 0.1 to 0.4 m on recoveries costing 9 to 46 m: nearly all of the degradation is removed by re-scaling one pre-flight scalar, the stall-speed normalization. The costliest errors are the pilot's, not the procedure's: hesitating one second before the pull costs 27 to 35 m, while the vigor of the pull is immaterial provided it is flown on the angle of attack.

*Ph.D. Candidate, Departamento de Ingeniería.

†Ph.D. Candidate, Departamento de Ingeniería.

‡Professor, Departamento de Ingeniería.

§Professor, Departamento de Ingeniería.

Nomenclature

ρ	air density
b	wing span
c	chord length
S	wing surface area
p_x, p_y, p_z	northward, eastward and down position
h	altitude, from the ground
u, v, w	body-x, y and z velocity
V	airspeed
α	angle of attack
β	sideslip angle
ϕ, θ, ψ	roll, pitch and yaw angles
γ	flight path angle
μ	bank angle
$\epsilon_0, \epsilon_1, \epsilon_2, \epsilon_3$	Euler parameters
p, q, r	roll, pitch and yaw rate
δ_e	elevator deflection, positive trailing edge down
δ_r	rudder deflection, positive trailing edge to the left
δ_a	aileron deflection, positive is trailing edge down of right aileron
δ_t	throttle position
$\hat{p}$	dimensionless roll rate, $\hat{p} = \frac{pb}{2V}$
L, D, Y	aerodynamic lift, drag and side force
M_x, M_y, M_z	aerodynamic rolling, pitching and yawing moment about the c.g.
C_L, C_D, C_Y	aerodynamic lift, drag and side force coefficient
C_l, C_m, C_n	aerodynamic rolling, pitching and yawing moment coefficient about the c.g.
f	system dynamic equation of motion
$V(\mathbf{x})$	value function
g	stage reward, non-positive during descent
a	vector of actions

I. Introduction

Loss of control in flight remains the largest single category of fatal aviation accidents [5], and its most common precursor is the aerodynamic stall: the wing driven past the angle of attack at which the flow can no longer stay attached, so that lift falls, drag rises, and the aircraft descends. Recovery is a two-phase maneuver, a nose-down input to restore attached flow followed by a pullout that arrests the descent, and its cost is measured in altitude: at pattern height, tens of meters separate a recovery from an accident.

For this maneuver the certified guidance is not mutually consistent. The U.S. Federal Aviation Administration (FAA) prescribes a sequential recovery, pitch nose-down first and power restored once the wing is unstalled [6], whereas the United Kingdom Civil Aviation Authority (CAA) commands full power *simultaneously* with the nose-down input [7]. Throughout this paper the two doctrines are named after their authorities, the *CAA sequence* and the *FAA sequence*, labels that name the power-timing skeletons as modeled here (the element on which the two publications genuinely differ) rather than the complete published procedures. A third tradition, deliberately delaying power until the pullout is complete (hereafter the *power-delayed* recovery), survives in training practice and was flown as a control arm in the only systematic empirical comparison to date, the fourteen-type flight-test campaign of Gratton et al. [4], which favored the simultaneous sequence (roughly two thirds of the height loss of the sequential one, which in turn cost about 90% of the deliberately delayed recovery). Flight test, however, can rank candidate procedures; it cannot say how far any of them stands from the optimum, because the optimum has never been available.

A parallel and equally unexamined gap exists in recovery *design*. Minimum-altitude-loss pullout and stall recovery are routinely posed as optimal control problems on reduced-order models of three or four states [2], under the implicit assumption that the omitted dynamics (pitch rate, sideslip, roll and yaw coupling) do not materially alter the optimal policy. This assumption is plausible but untested, because testing it requires the optimal policy of a higher-order model as a reference and the same computational obstacle has kept that reference out of reach. Making it computable is a direct consequence of the method presented here; the comparison itself belongs with the lateral dynamics and is deferred to the companion paper.

The obstacle is computational. Exact dynamic programming over a discretized state space scales exponentially with state dimension [8, 9]; for an eight-state spin model, even a coarse grid exceeds 10^{10} nodes [2], far beyond the memory available to tabular solvers that precompute and store transition tables. Practitioners have therefore relied on three alternatives, each with a structural limitation. Flight test and spin-tunnel experiments sample the envelope sparsely and at considerable cost and risk. Simplified analytic models trade fidelity for tractability [10, 11] and cannot represent the strongly coupled, post-stall regime in which spins develop. Deep reinforcement learning scales to high state dimension [12] but returns an approximate policy with no optimality certificate; its control boundaries are statistically blurred and cannot serve as ground truth against which other policies are measured.

This work removes the obstacle. We compute exact global optimal recovery policies by recasting the discretized dynamic program in a compute-bound form: rather than storing precomputed transition tables, the system dynamics are integrated on the fly within GPU registers, reducing the memory footprint from $O(N_s N_a 2^D)$ to $O(N_s)$ and bringing grids of up to $\sim 10^8$ states within reach of a single GPU. The interpolation that discretization forces is barycentric, hence non-expansive in the supremum norm, so the discretized Bellman backup retains the convergence of the exact one and the computed policy is the exact optimum of the discretized process [1, 13]. This formulation is summarized in Section II; it is the instrument of the present study rather than its subject.

This paper makes two contributions. *First*, the method: the compute-bound GPU formulation of exact dynamic programming, validated on the 3-DOF banked pullout of Bunge et al. [2] both qualitatively, reproducing the published value function and switching surfaces, and quantitatively, agreeing with a direct-collocation solution of the same optimal control problem to within 1.4% of the maximum altitude lost while resolving 31.9 million states in 15 minutes on a single GPU. *Second*, the first exact optimal stall-recovery policy computed on propulsion-coupled wind-tunnel aerodynamics: a 4-DOF symmetric stall model of a light general-aviation aircraft built on the tables of Riley [3], including his first-order engine lag with the time constant identified from his published throttle transients. The stall is *symmetric* by design, wings level and slip-free: the regime in which the certified procedures differ on power timing, and the one the flight-test comparison of Gratton et al. addressed. From this exact solution the simultaneous full-power sequence taught by the CAA *emerges* as the optimum for this model, the altitude cost of the delayed-power alternatives is

quantified across the stalled-entry set, and the pull is exposed as a sliding-mode arc riding the stall boundary. The policy remains effective across the aircraft’s achievable loading envelope, and re-solving it for the perturbed aircraft buys 0.1 to 0.4 m on recoveries costing 9 to 46 m: nearly all of the degradation is removed by re-scaling a single pre-flight scalar.

The question this framework is ultimately sized for lies one model beyond. For the developed spin, the PARE technique (power to idle, ailerons neutral, rudder opposite, elevator forward) is taught uniformly and embedded in flight manuals and type-certification guidance [6, 14], yet it was distilled from flight test, spin-tunnel campaigns, and accumulated pilot experience [15] rather than measured against a verified optimum, and posing that question requires an eight-state model. It is deferred to a companion paper; the present study establishes the method and answers the power-timing question one rung below.

The remainder of the paper is organized as follows. Section II presents the compute-bound GPU formulation, and Section III formalizes the minimum-altitude-loss recovery as an infinite-horizon optimal control problem with an absorbing set at level flight. Two case studies follow, each presenting its dynamic model and its results: Case I validates the solver on the 3-DOF banked pullout of Bunge et al. [2], and Case II scales the framework to the 4-DOF symmetric stall recovery with the propulsion-coupled aerodynamics of Riley [3], comparing the exact policy against the certified procedures flight-tested by Gratton et al. [4] and quantifying its robustness to the mass and CG variations of day-to-day operation.

II. Computational Method

A. The Memory Bottleneck of Tabular Dynamic Programming

Grid-based dynamic programming discretizes the continuous state and action spaces and iterates a Bellman backup over the resulting finite Markov decision process [8, 9]. Standard implementations precompute the dynamics once: for every state–action pair, the integrated next state is stored as the 2^D vertex indices and interpolation weights of the grid cell that contains it, so that the iterative sweeps reduce to table lookups. The stored tables occupy $O(N_s N_a 2^D)$ memory, where N_s is the number of states, N_a the number of actions, and D the state dimension. High-performance grid-based toolboxes accelerate the sweeps [16], but the tabular arrays they must hold in memory remain the binding constraint. The number is concrete: for the refined Case I grid of Sec. IV.D (31.9 million states, 91 actions, $D = 3$), the transition tables alone would occupy roughly 186 GB, beyond the memory of any single GPU, while the value function and policy arrays occupy about 0.25 GB.

B. In-Kernel Integration

The present solver stores no transition tables. During every sweep, each GPU thread is mapped to one state index, integrates the flight dynamics over the control interval with a fourth-order Runge–Kutta scheme held in registers, and evaluates the interpolated value of the resulting next state directly. The memory footprint falls to $O(N_s)$, the value and policy arrays themselves, at the price of recomputing the dynamics on every sweep. Trading stored transitions for arithmetic is a standard move in GPU computing; what makes it worth stating here is that the recomputed backup is identical to the tabular one (Sec. II.C), and that the arithmetic lands where the hardware is strongest: the profiling of Table 2 shows the solve remains compute-bound at scale, with policy evaluation accounting for 99.6% of the runtime and 41 ms of device-to-host traffic in a 15-minute run.

One implementation detail is essential for throughput. Locating the grid cell that contains the next state, and weighting its vertices, is naturally written with conditional logic, which makes threads within a CUDA warp diverge. The kernel instead uses a branchless formulation. Let x'_j be the j -th coordinate of the integrated next state and

$$n_j = \frac{x'_j - x_j^{\min}}{x_j^{\max} - x_j^{\min}} (N_j - 1) \quad (1)$$

its fractional grid index, where N_j is the number of nodes of dimension j . For each vertex $\mathbf{v} \in \{0, 1\}^D$ of the enclosing

cell, the barycentric weight is

$$W_{\mathbf{v}} = \prod_{j=1}^D [v_j \cdot \delta_j + (1 - v_j)(1 - \delta_j)], \quad \delta_j = n_j - \lfloor n_j \rfloor, \quad (2)$$

so that all threads in a warp execute the same instruction stream regardless of where their states land. In what follows, $\mathcal{V}(s')$ denotes the set of the 2^D vertices of the cell enclosing s' , so the interpolated value of a next state reads $\sum_{\mathbf{v} \in \mathcal{V}(s')} W_{\mathbf{v}} V(\mathbf{v})$.

C. Interpolation Preserves the Convergence Guarantee

Discretizing the state space forces the value function to be evaluated between grid nodes, and the choice of interpolator determines whether the discretized backup keeps the convergence properties of the exact one. The scheme above is the barycentric framework of Munos and Moore [1]: because the weights of Eq. (2) are non-negative and sum to one, the interpolation operator $\mathcal{I}$ is *non-expansive* in the supremum norm,

$$\|\mathcal{I}V - \mathcal{I}W\|_{\infty} \leq \|V - W\|_{\infty},$$

so the interpolated backup inherits the contraction structure of the exact operator instead of amplifying approximation error, and the discretized problem is consistent with the continuous one as the mesh is refined [1, 13].

This is the point of the architecture. Integrating the dynamics inside the kernel does not modify the mathematical operator at all: each thread computes the same next state, the same 2^D vertices, and the same convex weights that a tabular implementation would have precomputed, so the $O(N_s)$ footprint costs nothing in the mathematics. The complexity reduction and the preservation of the guarantee are therefore obtained together, and it is the combination that matters: the guarantee is what makes the computed policy a trustworthy optimum, and the reduced footprint is what lets the grid that carries the guarantee be fine enough to resolve the recovery.

Two properties of the present formulation should be stated explicitly, since they determine what the converged solution certifies. First, the recursion is *undiscounted* (unit discount factor): the objective is the total altitude change to absorption, and no discount factor is introduced, so the operator is non-expansive rather than a strict contraction. Convergence rests instead on the stochastic-shortest-path structure of the problem [9], and the discretized process belongs to that class exactly, not approximately: the barycentric weights, non-negative and summing to one, are formally transition probabilities, so the interpolated deterministic dynamics define a genuine stochastic shortest path over the grid nodes [1]. Its hypotheses are met:

- The level-flight set $\{\gamma \geq 0\}$ and the departure set are absorbing and cost-free thereafter, so the value on both is known exactly.
- Every state admits at least one policy that reaches one of them.
- The stage reward is non-positive throughout the descent, so a policy that never reaches either terminal set accumulates unbounded loss and is excluded by the maximization.

Second, the object that converges is the optimal value function of the *discretized, interpolated, finite-action* Markov decision process induced by the grid, the action set and the control interval. Optimality is exact for that process; its distance to the continuous optimum is a discretization question, addressed for Case I in Sec. IV.D by refinement.

D. The Control Interval Δt

The integration interval Δt appearing in Algorithm 1 is a first-class solver parameter rather than a fixed constant of the method, because its value couples directly to the spatial grid through the barycentric backup. If Δt is chosen so small that the RK4 displacement covers only a small fraction of a grid cell, the interpolation weights of Eq. (2) concentrate on the origin vertex: the state effectively transitions to itself, value information propagates only a fraction of a cell per sweep, and near-ties between actions are amplified in the arg max. If Δt is too large, the one-step transition skips over cells, degrading the consistency of the multilinear backup and the detection of terminal-set crossings. The natural scale is a Courant–Friedrichs–Lewy (CFL) type condition [17]: the displacement per backup should be commensurate with

one cell, $\Delta t^* \approx (\sum_i |\dot{x}_i|/h_i)^{-1}$, where h_i is the grid spacing of dimension i .

The kernel accordingly decouples the two roles the interval plays. The *control* interval, over which the action is held and the Bellman backup is applied, is a configurable parameter, selected per case study against the CFL-like scale above and reported with each discretization; internally it is subdivided into RK4 *micro-steps*, so that integration accuracy and terminal-event detection (success and crash crossings are checked at micro-step resolution) are independent of the backup interval. The solver additionally supports a state-endogenous variant that evaluates $\Delta t(s) = \min\{\Delta t_{max}, (\sum_i |\dot{x}_i(s)|/h_i)^{-1}\}$ on the fly inside the kernel, at no memory cost, since the derivatives are already computed for the RK4 stage; the case studies reported here use the fixed-interval configuration.

E. Policy Iteration

Where prior work in this domain used Value Iteration [2], this solver implements Policy Iteration, summarized in Algorithm 1. Policy Iteration requires a full policy-evaluation step but typically converges in far fewer iterations, moving between vertices of the policy polytope rather than approaching the fixed point linearly [9]. The evaluation step, traditionally the bottleneck of the method, is here an iterative sweep over the entire state array in a single kernel launch per pass, so its cost is the per-sweep cost already profiled in Table 2. One safeguard deserves note: the evaluation is run to a tolerance rather than to exactness, and the value function is initialized at zero, an upper bound under non-positive rewards. The scheme is therefore a modified policy iteration [9], whose convergence under the conditions of Sec. II.C does not require the initial random policy to be proper; a policy that traps part of the state space merely sinks the value there sweep by sweep until the next improvement replaces it.

Algorithm 1 Massively Parallel Policy Iteration for High-Fidelity Grids

```

1: Initialize:  $V(s) \leftarrow 0$  and  $\pi(s) \leftarrow \text{random}$  for all  $s \in \mathcal{S}$ , the set of the  $N_s$  grid states
2:  $policy\_stable \leftarrow \text{false}$ 
3: while not  $policy\_stable$  do
4:   {Step 1: Parallel Policy Evaluation}
5:   repeat
6:      $\Delta \leftarrow 0$ 
7:     for all thread  $s \in \mathcal{S}$  in parallel do
8:        $V_{old} \leftarrow V(s)$ 
9:        $s' \leftarrow \text{RK4}(s, \pi(s), \Delta t)$ 
10:       $V(s) \leftarrow g(s, \pi(s)) + \sum_{\mathbf{v} \in \mathcal{V}(s')} W_{\mathbf{v}} V(\mathbf{v})$  { $\mathcal{V}(s')$ : vertices of the cell containing  $s'$ ; weights from Eq. (2)}
11:       $\Delta \leftarrow \max(\Delta, |V_{old} - V(s)|)$ 
12:     end for
13:   until  $\Delta < \text{tolerance}$ 
14:   {Step 2: Parallel Policy Improvement}
15:    $policy\_stable \leftarrow \text{true}$ 
16:   for all thread  $s \in \mathcal{S}$  in parallel do
17:      $old\_action \leftarrow \pi(s)$ 
18:      $\pi(s) \leftarrow \arg \max_{a \in A} [g(s, a) + \sum_{\mathbf{v} \in \mathcal{V}(s')} W_{\mathbf{v}} V(\mathbf{v})]$ , with  $s' = \text{RK4}(s, a, \Delta t)$ 
19:     if  $old\_action \neq \pi(s)$  then
20:        $policy\_stable \leftarrow \text{false}$ 
21:     end if
22:   end for
23: end while

```

III. Problem Formulation

The minimum-altitude-loss pullout is naturally posed as a free-final-time optimal control problem with a terminal constraint, Eq. (3): find the control history that steers the aircraft from the upset state x_0 to level flight, $\gamma(t_f) = 0$, while

minimizing the altitude lost along the way. Problems of this form can be solved by indirect methods based on the calculus of variations, or by direct methods such as collocation-based trajectory optimization. Either way, the solution is an open-loop control history valid for a single initial condition: recovering from a different upset requires solving the problem anew.

$$h^*(x_0) = \min_{a(\cdot), t_f} \int_0^{t_f} -V \sin \gamma \, dt \quad \text{s.t.} \quad \dot{x} = f(x, a), \, x(0) = x_0 \text{ and } \gamma(t_f) = 0 \quad (3)$$

Two properties of Eq. (3) permit a stronger result. The running cost $-V \sin \gamma$ depends only on the state, and it vanishes on the terminal manifold $\gamma = 0$. Consequently, if the level-flight set $\{\gamma \geq 0\}$ is made absorbing (freezing the dynamics once the flight-path angle reaches zero), the trajectory accumulates no further cost after recovery, and the problem becomes the infinite-horizon formulation of Eq. (4). Since $\gamma(t)$ is continuous, the absorbing set is entered exactly at $\gamma = 0$, where the running cost vanishes, so the integral remains finite.

$$h^*(x_0) = \min_{a(\cdot)} \int_0^\infty -V \sin \gamma \, dt \quad \text{s.t.} \quad \begin{cases} \dot{x} = f(x, a), & \gamma < 0 \\ 0, & \gamma \geq 0 \end{cases} \quad (4)$$

$$x(0) = x_0$$

The value function of Eq. (4) satisfies a stationary Bellman equation, which dynamic programming solves for all initial conditions simultaneously: the solution is no longer a single trajectory but the optimal feedback policy $a^* = \pi^*(x)$, yielding the minimum-altitude-loss recovery from every state in the flight envelope. The solver implements this minimization as the equivalent maximization of a non-positive stage reward: $g(s, a)$ in Algorithm 1 accumulates the (negative) altitude change over one control interval, the negative of the running cost of Eq. (4), so the arg max of the backup and the minimization of Eq. (4) select the same actions.

IV. Case I: 3-DOF Banked Pullout Validation against Established Literature

A. Reduced-Order Equations of Motion

Following Bunge et al. [2], the banked pullout is modeled as a three-state point mass under four simplifying assumptions: the lift coefficient and the bank rate are tracked by high-bandwidth inner loops (elevator and aileron, respectively) whose dynamics are neglected; sideslip remains zero; drag depends on the state only through the lift coefficient; and power is at idle. The state vector is $\mathbf{x} = [\gamma, V/V_s, \mu]^\top$ (flight path angle, normalized airspeed, and bank angle), and the control vector collects the two commanded quantities, $\mathbf{a} = [C_L^{cmd}, \dot{\mu}_{cmd}]^\top$:

$$\dot{\gamma} = \frac{\rho S}{2m} V C_L^{cmd} \cos \mu - \frac{g \cos \gamma}{V}, \quad (5)$$

$$\dot{V} = -g \sin \gamma - \frac{\rho S}{2m} V^2 C_D(C_L^{cmd}), \quad (6)$$

$$\dot{\mu} = \dot{\mu}_{cmd}. \quad (7)$$

Equations (5)–(6) are written in the dimensional airspeed V ; the solver stores and integrates the normalized state component V/V_s , whose rate is $\dot{V}/V_s$. The drag coefficient is a static function of the commanded lift, obtained by inverting the linear lift curve $\alpha = (C_L - C_{L_0})/C_{L_\alpha}$ and evaluating the quadratic polar $C_D = C_{D_0} + C_{D_\alpha} \alpha + C_{D_{\alpha^2}} \alpha^2$ ($C_{L_0} = 0.41$, $C_{L_\alpha} = 4.70 \text{ rad}^{-1}$, $C_{D_0} = 0.0525$, $C_{D_\alpha} = 0.2068$, $C_{D_{\alpha^2}} = 1.8712$). The aircraft is the same Grumman American AA-1 Yankee of Case II, at the flight weight used by [2] ($m = 697.18 \text{ kg}$, $S = 9.1147 \text{ m}^2$).

The control bounds encode the physics that the reduced model does not resolve. The commanded lift is limited to $C_L^{cmd} \in [-0.5, 1.0]$, a margin of 0.2 below the positive stall value ($C_L = 1.2$) and above the negative (inverted) stall value ($C_L \approx -0.7$), so that commanded-lift trajectories never enter the separated-flow regime the model cannot represent. The bank rate is limited to $|\dot{\mu}_{cmd}| \leq 30 \text{ deg/s}$, the steady-state roll rate with full aileron at stall speed [2]. This commanded- C_L , commanded-bank-rate abstraction is precisely the perfect-inner-loop assumption that Case II

removes by resolving the pitch dynamics explicitly.

B. Baseline Discretization Benchmarking

For validation against the published benchmark, the state-control space was first discretized on the rectangular grid of Bunge et al. [2]. The flight dynamics are integrated with a fixed time step of $\Delta t = 0.1$ seconds. The stage reward $g(s, a)$ is the altitude change over the interval, non-positive during the descent, minus a quadratic penalty that ensures the smoothness of the bank rate command policy, preventing jagged control surfaces: the smoothing device introduced by Bunge et al. [2], here with a lighter weight (0.001 vs. their 0.01) that preserves the regularizing role while keeping the altitude-loss term strictly dominant:

$$g(s, a) = V \sin \gamma \Delta t - 0.001 \dot{\mu}_{cmd}^2 \quad (8)$$

The baseline grid boundaries and increments, identical to those of Bunge et al., are reproduced in Table 7 (Appendix A); Sec. IV.D refines them.

On this baseline grid the full Policy Iteration solve is essentially instantaneous: Table 8 (Appendix A) breaks down the wall-clock cost as measured with CUDA events on an NVIDIA GeForce RTX 4090. The profile is the compute-bound architecture of Section II made visible. Policy evaluation (the phase that traditional implementations amortize with precomputed transition tables) accounts for 98.5% of the runtime and executes entirely on the GPU at 12 ms per sweep; policy improvement, which evaluates all 91 actions per state, costs 0.16 ms per iteration; and the total device-to-host traffic is 0.2 ms. Solving the benchmark problem of [2] to full convergence takes 0.38 s end to end.

C. Baseline Validation and Optimal Value Function Topology

Upon convergence on the baseline grid, the optimal value function $V^*(s)$ maps each state to the minimal altitude loss achievable from it. Because the stage reward $g(s, a)$ is defined primarily by the incremental altitude change, the total altitude loss Δh_{min} required to return the aircraft to level flight ($\gamma = 0^\circ$) can be extracted directly from the converged grid:

$$\Delta h_{min}(V_0, \gamma_0, \mu_0) \approx -V^*(V_0, \gamma_0, \mu_0) \quad (9)$$

where the approximation absorbs the small accumulated contribution of the bank-rate penalty of Eq. (8). The converged value function was queried across slices of constant initial normalized airspeed ($V_0/V_s \in \{1.2, 2.0, 3.0, 4.0\}$). The resulting altitude-loss contours over the initial bank and flight-path envelope (Fig. 10, Appendix A) reproduce the global topology published by Bunge et al. [2], validating the accuracy of the on-the-fly RK4 integration and the consistency of the barycentric interpolator.

D. Scaling the Grid: The Practical Reach of the Compute-Bound Architecture

The baseline grid replicates the published benchmark at interactive speed, but its 53,280 states are coarse. To probe the practical reach of the architecture, the same problem was re-solved on a grid refined in every state dimension: 31,948,500 states (a 600-fold increase) with the same 91 actions. Table 2 reports the cost on the same GPU: full Policy Iteration converges in 67 iterations and 14.9 minutes. Two features of the profile deserve note. First, the solve remains strictly compute-bound at scale: policy evaluation accounts for 99.6% of the runtime, and the total device-to-host traffic is 41 ms of a 15-minute run. Second, the per-sweep cost grows close to linearly with the state count (12.1 ms to 13.3 s per evaluation sweep, a factor of 1100 for 600 $\times$ the states; the residual 1.8 $\times$ is occupancy and memory-traffic overhead at the larger grid, not an extra order of complexity), consistent with the $\mathcal{O}(N_s)$ footprint of Section II.

The curse of dimensionality is not defeated (work and memory still scale exponentially with the state dimension), but the constant in front of that exponential is now small enough that grids three orders of magnitude beyond the published state of the art [2] fit on a single workstation GPU. The refined grid is detailed in Table 1; the policies it produces (Fig. 1) are examined next.

Table 1 Discretization of the refined state-control space: the grid of Fig. 1 and Tables 2 and 3. The baseline grid of Bunge et al. [2] it refines is reproduced in Table 7, Appendix A.

Variable	Lower Bound	Upper Bound	Points	Increment	Units
γ	-180	0	361	0.5	deg
V/V_s	0.9	4.0	250	~ 0.0124	—
μ	-20	200	354	~ 0.623	deg
C_L^{cmd}	-0.5	1.0	7	0.25	—
$\dot{\mu}_{cmd}$	-30	30	13	5	deg/s

$361 \times 250 \times 354 = 31,948,500$ states; $7 \times 13 = 91$ actions.

E. Optimal Control Policy: Switching Surfaces and Asymmetry

The optimal policy $\pi^*(s)$ extracted from the converged value function exhibits a *bang-bang* control architecture. This behavior is characteristic of shortest-path optimal control problems, where applying maximum allowable control authority at all times drives the system to the terminal state (level flight) with minimal accumulated cost [2].

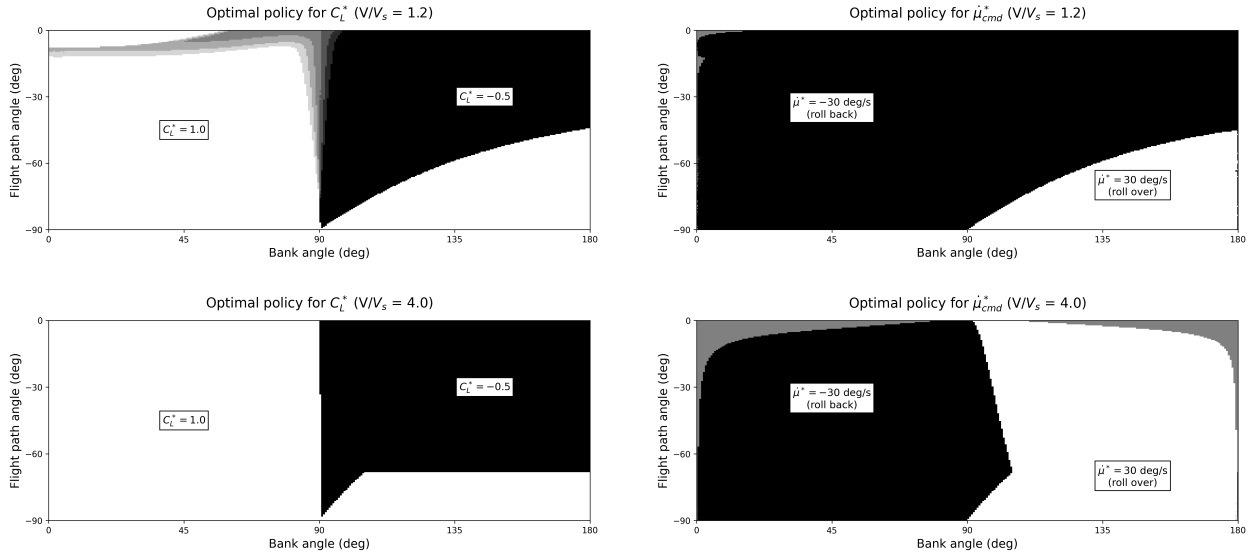


Fig. 1 Optimal policy on the 31.9-million-state grid: commanded lift coefficient (C_L^* , left column) and bank rate ($\dot{\mu}_{cmd}^*$, right column) at $V/V_s = 1.2$ (top row) and $V/V_s = 4.0$ (bottom row). The boundaries between the black and white regions are the control switching surfaces; the baseline-grid maps these refine are shown in Fig. 11, Appendix A.

Figure 1 maps the optimal commanded lift (C_L^*) and bank rate ($\dot{\mu}_{cmd}^*$) over the bank and flight-path envelope at $V_0/V_s = 1.2$ and 4.0 , the product of the 14.9-minute solve profiled in Table 2.

Table 2 Computational cost of Policy Iteration on 31,948,500 states, 91 actions (NVIDIA GeForce RTX 4090).

Phase	Total (ms)	Per Iteration (ms)
State grid construction (CPU)	32.25	—
Policy Evaluation (GPU)	891257.49	13302.35
Policy Improvement (GPU)	3232.20	48.24
GPU $\rightarrow$ CPU Transfer	41.33	—
Total	894563.27	—

Converged in 67 iterations (~ 14.9 min). Timing via CUDA events.

Away from the switching surfaces (the boundaries where the command flips from one extreme to the other) the actions saturate at their bounds, $C_L^* \in \{-0.5, 1.0\}$ and $\mu_{cmd}^* \in \{-30, 30\}$ deg/s, with only a thin band of near-tied intermediate commands along the surfaces themselves. Compared with the baseline-grid maps (Fig. 11, Appendix A), the surfaces sharpen from stair-stepped bands into smooth curves while the global structure is unchanged. The notable feature of the policy is the asymmetry of the surfaces. The bank-rate switching boundary does not lie at the 90° knife-edge but well into the inverted regime: the lift bounds are asymmetric (1.0 up against -0.5 down [2]), pulling up is twice as effective as pushing down, and the optimum therefore rolls back toward upright even when banked past 90° . Committing instead to an inverted pullout ($+30$ deg/s, $C_L^* = -0.5$) pays only in deep, highly banked dives, where rolling back upright costs more time than the inverted recovery costs aerodynamically; and that region shrinks with airspeed, since at $V/V_s = 4.0$ the aircraft has the energy for its tightest turn and the upright $C_L = 1.0$ recovery wins over a broader range of the envelope.

F. Trajectory-Level Validation Against Direct Optimal Control

A second, independent check quantifies the solver against continuous-time optimal control: for each initial condition of Figures 3 and 4 of [2] (the two sweeps of Table 3), the free-final-time problem of Eq. (3) was solved by direct collocation [18] (CasADi [19] with IPOPT [20]), warm-started with the DP rollout, since the nonconvex NLP needs an initialization in the correct basin. Their agreement is a bidirectional check: the NLP confirms the DP’s local optimality, and the DP certifies that the NLP descended the global basin. The two objects also differ in kind: the NLP decides one open-loop control history per entry, all node controls jointly, re-solved for each initial condition; the DP is a feedback law over the entire envelope, optimal again from any perturbed state.

Table 3 gives the number on the refined grid of Sec. IV.D; the trajectory pairs are reproduced in Fig. 12, Appendix A. The metric, the gap over the deepest loss of each sweep, is at most 1.4% (0.9% on average, 2.1 m at worst); the qualitative structure of [2] is reproduced, including the inverted pullout at the near-inverted entry. The continuous solution is the better of the two at every entry, as it must be: every discretized policy is a feasible solution of the continuous problem, so its optimum can only match or beat the DP; a reversed sign would indict the NLP’s convergence, not the method.

Table 3 Closed-loop DP rollouts vs. DP-warm-started CasADi/IPOPT solutions of Eq. (3) at the initial conditions of Bunge et al. [2] ($V_0/V_s = 1.2$); both end at the level-flight recovery $\gamma = 0$, and e is the gap over the deepest loss of each sweep. The $\gamma_0 = -60^\circ$, $\mu_0 = 150^\circ$ entry is excluded (not returned to level flight).

γ_0 (deg)	μ_0 (deg)	DP (m)	NLP (m)	gap (m)	e (%)
-30	150	-157.38	-156.34	-1.04	0.67
-30	120	-126.33	-125.31	-1.02	0.65
-30	90	-90.00	-88.33	-1.67	1.07
-30	60	-62.28	-60.14	-2.14	1.37
-30	30	-50.40	-49.44	-0.96	0.61
-60	120	-194.89	-193.14	-1.75	0.91
-60	90	-152.31	-150.61	-1.70	0.88
-60	60	-121.14	-119.48	-1.66	0.86
-60	30	-109.42	-107.76	-1.66	0.86
Mean over the nine entries				-1.51	0.88

The gap splits into an accounting artifact and a structural price. The artifact: the evaluation rollout accumulates altitude with forward Euler at $\Delta t = 0.1$ s, a first-order, one-signed error (the NLP’s collocation error is orders smaller) that telescopes to $(\Delta t/2) V_0 |\sin \gamma_0|$ and matches the smallest gap of each sweep to three figures (0.96 m at $\gamma_0 = -30^\circ$, 1.66 m at -60°). The structural price, at most 1.2 m: the NLP, free to pick any control value at any instant, simply selects the optimal one, while the discrete policy can realize an intermediate control only by alternating its 91 actions at the 0.1 s hold; since the optimum is nearly bang-bang, both fly the same extremes for most of the maneuver and the price concentrates at the switch times. Tightening the NLP would lower its own losses and widen the gap, not close it.

V. Case II: 4-DOF Symmetric Stall Recovery with Riley (1985) Aerodynamics

This section poses the question the paper was built to answer: flown optimally, does the symmetric stall recovery apply power at once or delay it, and what does either choice cost? The 3-DOF pullout of Sec. IV validated the solver but could not ask this: its controls were the outputs of an assumed-perfect inner loop (commanded lift and bank rate), and it had no throttle. Here the framework is applied to a four-state longitudinal model that resolves what the pilot actually commands. The state vector is $\mathbf{x} = [\gamma, V/V_s, \alpha, q]^T$ (flight path angle, normalized airspeed, angle of attack, and pitch rate) and the control vector is $\mathbf{u} = [\delta_e, \delta_t]^T$ (elevator deflection and throttle position): the pitch dynamics are resolved rather than assumed tracked, and the timing of power application becomes a decision variable of the optimization instead of an input from doctrine. The aerodynamics come from Riley’s propulsion-coupled wind-tunnel tables, measured through stall and post-stall at two propulsive conditions, and the propulsion carries his first-order engine lag; both are developed in the following subsections. The physical parameters correspond to the Grumman American AA-1 Yankee as tabulated by Riley [3] and are collected in Table 9 of Appendix B, together with the derived reference quantities used throughout this section, in particular the stall speed $V_s \approx 31.6$ m/s that normalizes the airspeed state.

The model is deliberately symmetric: wings level, zero sideslip, no lateral-directional states. Real stalls frequently depart asymmetrically (a wing drop at the break is the precursor of the incipient spin), and no longitudinal model can speak to that phase. The scope of this section is therefore the symmetric stall proper: the regime in which the certified procedures disagree on power timing, and the one the flight-test comparison of Gratton et al. [4] addressed.

A. Equations of Motion

The longitudinal dynamics are

$$\dot{\gamma} = \frac{L}{mV} - \frac{g \cos \gamma}{V}, \quad (10)$$

$$\dot{V} = -g \sin \gamma - \frac{D}{m}, \quad (11)$$

$$\dot{\alpha} = q - \dot{\gamma}, \quad (12)$$

$$\dot{q} = \frac{M_y}{I_{yy}} = \frac{\bar{q} S c C_m}{I_{yy}}, \quad (13)$$

with $L = \bar{q} S C_L$, $D = \bar{q} S C_D$, $\bar{q} = \frac{1}{2} \rho V^2$, and the dimensionless pitch rate $\hat{q} = qc/(2V)$ entering the aerodynamic coefficients below. As in Sec. IV, the equations are written in the dimensional airspeed V while the solver stores and integrates the normalized V/V_s .

Notably, Eqs. (10)–(11) contain *no explicit thrust terms* ($T \sin \alpha$, $T \cos \alpha$). This is not an approximation but a direct consequence of how Riley [3] defines the power-on aerodynamic data. The wind-tunnel measurements at the powered condition were conducted with a running propeller installed on the airframe, so the force balance measured the *net axial force* acting on the complete propulsion–airframe system. Normalized by $\bar{q} S$, the tabulated drag entry is therefore a net axial-force coefficient,

$$C_{D_o}|_{C_T=0.5} = \frac{D_{\text{aero}} - T}{\bar{q} S} = C_{D,\text{aero}} - C_T, \quad (14)$$

where the thrust coefficient is defined as

$$C_T = \frac{T}{\bar{q} S}. \quad (15)$$

The negative values of the power-on drag column at low angles of attack (Table 13: $C_{D_o} = -0.347$ at $\alpha = 0^\circ$) provide direct empirical confirmation: aerodynamic drag is intrinsically positive, so a negative tabulated value can only arise when the propeller thrust dominates, yielding a net forward force. Inserting an explicit thrust term into Eq. (11) while also using the Riley power-on tables would therefore *double-count* the propulsive contribution. Thrust enters the model exclusively through C_T , which selects the operating point of the aerodynamic tables. The thrust itself follows Riley’s Appendix A: the cockpit throttle δ_t maps to the intermediate setting $\delta'_t = 0.65 \delta_t + 0.35$ of his Eq. (A3), and sea-level thrust is interpolated from the twelve-entry table of his Eq. (A9), $T = T_0(\delta'_t) + T_1(\delta'_t) V$: about 2.0 kN at full

throttle statically, decaying with airspeed. Definition (15) couples the propulsive and aerodynamic states: as the aircraft decelerates during the stall, $\bar{q}$ collapses and C_T rises for a fixed throttle, shifting the interpolated coefficients toward the power-on tables.

One element of the propulsion model is deliberately left out of the dynamic program: Riley’s Eq. (A4) places a first-order lag, $\tau_e \dot{\delta}_t = \delta_{t,c} - \delta_t$, between the commanded and the effective throttle. Riley defines τ_e and never tabulates its value; digitizing his published throttle transients and fitting the lag through his Eqs. (A3) and (A12) recovers $\tau_e = 0.8 \pm 0.2$ s, and $\tau_e = 0.85$ s is used throughout. Carrying the lag into the dynamic program would promote the effective throttle to a fifth state and multiply the grid: eleven bins take the 14.9 million states of Sec. V.C to 164 million, and the solve from hours to days. The policy is therefore solved against an ideal engine, and every trajectory in this section is flown through the lagged one; Sec. V.E quantifies what the lag costs and the sensitivity of the conclusions to τ_e .

B. Propulsion-Coupled Aerodynamic Model from Wind-Tunnel Data

The aerodynamic model replaces the constant-derivative formulations of prior stall-recovery studies [12] with the complete tabulated aerodynamics of Riley [3] (NASA TM-86309): not a fitted model but measurements, from full-scale powered wind-tunnel tests of the aircraft type in the Langley 30- by 60-Foot Tunnel with a thrust-torque balance between engine shaft and propeller. Seven coefficient tables are extracted directly from his Table III: the baseline lift, axial-force, and pitching-moment coefficients (C_{L_o} , C_{D_o} , C_{m_o}), the pitch-rate derivatives ($C_{L\dot{q}}$, $C_{m\dot{q}}$), and the elevator effectiveness derivatives ($C_{L\delta_e}$, $C_{m\delta_e}$), each on a common 14-point angle-of-attack grid from -10° to 40° , at power-off ($C_T = 0$) and a representative power-on condition ($C_T = 0.5$). The complete transcribed tables are collected in Appendix B, Fig. 2 plots the four that drive the recovery physics.

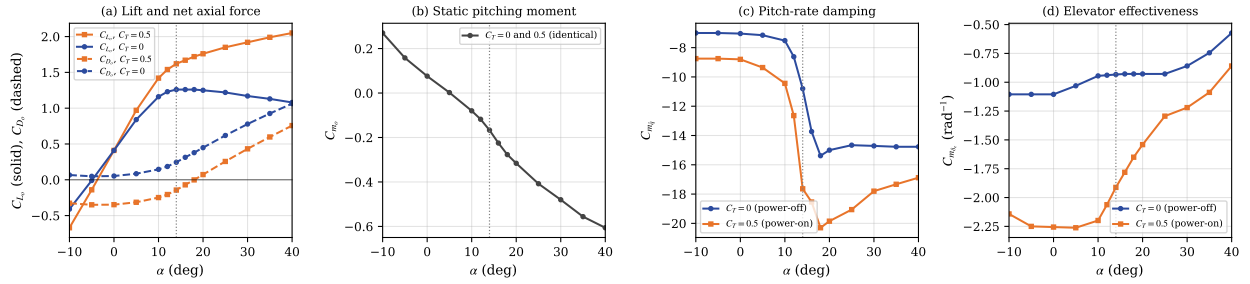


Fig. 2 The Riley (1985) coefficients that drive the recovery, at the two tabulated propulsive conditions (complete tables in Appendix B). (a) Baseline lift and net axial force: the power-off flat-top stall plateau against the monotone power-on growth of slipstream augmentation, and the negative power-on axial force that fingerprints the embedded thrust of Eq. (14). (b) The static pitching moment is C_T -independent. (c) Pitch-rate damping more than doubles across the stall. (d) Elevator moment effectiveness. The dotted line marks the power-off stall angle $\alpha_s = 14^\circ$.

Three features of the data drive the results of this section, and none survives a constant-derivative approximation. First, the power-off lift curve shows the characteristic flat-top plateau ($C_{L_o} = 1.26$ between 14° and 18°) while the power-on curve keeps *increasing* up to 40° : the propeller slipstream raises the dynamic pressure over the wing, so powered lift remains available deep into the post-stall regime. The plateau onset defines the reference stall angle $\alpha_s = 14^\circ$; it is a power-off boundary, and the policy will be seen to exploit the powered lift beyond it. Throughout this paper, *stall boundary* refers to this power-off angle: under power the aerodynamic stall itself moves, but α_s remains the reference the recovery is measured against. Second, the power-on axial-force column is negative below $\alpha \approx 18^\circ$: the empirical fingerprint of the embedded thrust of Eq. (14). Third, the pitch-rate damping $C_{m\dot{q}}$ more than doubles across the stall, from -7.15 at cruise to -15.4 power-off (-20.3 power-on).

The slipstream increment is aerodynamic, not mechanical: an explicit thrust term projects a force but leaves the coefficients untouched, so decoupled models systematically underpredict the normal force available during a powered

recovery. Altitude-loss figures published on such models [12] are therefore not commensurable with the results below, and no quantitative comparison is attempted; the validation of the framework rests on Case I, where benchmark and solver share the same plant.

Each coefficient is evaluated bilinearly: a linear table lookup along the α axis at each of the two tabulated C_T levels, blended linearly in C_T , with the lookup clipped to the tabulated range $C_T \in [0, 0.5]$. The clip applies to the table lookup only, not to the thrust itself: in the recovery regime C_T routinely exceeds the tabulated 0.5 (full throttle at $0.57 V_s$ gives $C_T \approx 1$), and the excess axial force is restored through Riley’s Appendix B correction, $\Delta C_{D_T} = -0.80 (C_T - 0.5) \cos \alpha$ for $C_T > 0.5$ and $-0.80 C_T \cos \alpha$ for $C_T < 0$, identically zero inside the tabulated range. The lift and moment columns, by contrast, saturate at the $C_T = 0.5$ measurement: the slipstream augmentation beyond the tabulated condition is not extrapolated, a conservative choice. The complete aerodynamic model is then

$$C_L = C_{L_o}(\alpha, C_T) + C_{L_{\delta_e}}(\alpha, C_T) \delta_e + C_{L_{\dot{q}}}(\alpha, C_T) \dot{q} + C_{L_{\dot{\alpha}}}(\alpha, C_T) \dot{\alpha}, \quad (16)$$

$$C_D = C_{D_o}(\alpha, C_T) + C_{D_{\delta_e}}(\alpha, C_T) \delta_e + C_{D_{\delta_e^2}}(\alpha, C_T) \delta_e^2, \quad (17)$$

$$C_m = C_{m_o}(\alpha, C_T) + C_{m_{\delta_e}}(\alpha, C_T) \delta_e + C_{m_{\dot{q}}}(\alpha, C_T) \dot{q} + C_{m_{\dot{\alpha}}}(\alpha, C_T) \dot{\alpha} + \frac{\Delta x_{cg}}{\bar{c}} C_L, \quad (18)$$

with $\dot{\alpha} = \dot{\alpha} c / (2V)$ the dimensionless rate of change of angle of attack, formed like $\dot{q}$.

Every longitudinal coefficient tabulated by Riley is used. The elevator drag terms $C_{D_{\delta_e}}$ and $C_{D_{\delta_e^2}}$ matter because the elevator is deflected throughout a recovery, and the quadratic term gives the nose-down phase an interior optimum rather than an actuator-limited one. The rate derivatives $C_{L_{\dot{\alpha}}}$ and $C_{m_{\dot{\alpha}}}$ are reported separately from $C_{\dot{q}}$ by Riley — many models lump the two into a single damping term — and both *change sign at the stall*: $C_{m_{\dot{\alpha}}}$ goes from -0.38 at $\alpha = 12^\circ$ to $+1.80$ at 14° , so it damps below the stall and anti-damps above it. Omitting them is therefore not uniform over the domain this maneuver traverses.

Because C_L depends on $\dot{\alpha}$ while Eq. (12) makes $\dot{\alpha}$ depend on $\dot{\gamma}$, which Eq. (10) in turn obtains from C_L , the rate terms close an implicit loop. It is resolved algebraically in closed form rather than by fixed-point iteration, so that the CUDA kernel and the CPU reference evaluate the identical expression; the correction is small, the implicit denominator staying within 1 ± 0.005 over the whole speed range of the grid.

The last term of Eq. (18) transfers the pitching moment to a center of gravity displaced Δx_{cg} aft of the Riley reference ($0.25 \bar{c}$); $\Delta x_{cg} = 0$ is used throughout, and the robustness study of Sec. V.F exercises the term. All lookups execute inside the CUDA kernel, keeping the solver compute-bound.

C. Discretization, Stage Cost, and Solver Configuration

The stage reward retains the pure altitude-loss structure of the infinite-horizon formulation,

$$g(s, a) = V \sin \gamma \Delta t, \quad (19)$$

stated here in reward (maximization) form, the negated altitude term of the Case I cost, with the absorbing success set $\{\gamma \geq 0\}$. Two deliberate differences from the Case I objective of Eq. (8) follow from the higher model fidelity. First, the quadratic bank-rate smoothing penalty inherited from [2] has no counterpart here. In the reduced-order model the bank rate is a free kinematic input, so infinitely many roll histories tie and the penalty stands in for the unmodeled actuation. Here the actuation is modeled: the elevator acts through the moment chain $\delta_e \rightarrow C_m \rightarrow \dot{q} \rightarrow q \rightarrow \dot{\alpha}$, whose inertia and stall-amplified pitch damping (Fig. 2c) low-pass filter the command. Residual action ties on the constraint arc still produce bang-bang switching in the raw policy map (the discrete approximation of the singular-arc equivalent control, Sec. V.E), but the plant dynamics absorb it: $\alpha(t)$ rides the stall boundary smoothly, and the closed-loop altitude loss is insensitive to command smoothing within 0.3 m. The objective remains pure altitude loss. Second, unlike Case I, whose commanded lift is bounded within safe limits, the 4-DOF model resolves the post-stall aerodynamics and can genuinely crash. The catastrophic terminal set $\{|\alpha| \geq 40^\circ\} \cup \{\gamma \leq -\pi + 0.05\}$ (angle-of-attack departure beyond the wind-tunnel data, or a near-vertical inverted dive) carries a fixed penalty of $-1000 V_s$. Its $\alpha = +40^\circ$ branch is the upper edge of the grid, where the penalty is pre-assigned to the terminal cells and propagates inward through the multilinear

backup; the negative branch and the inverted dive lie outside the grid and are detected during integration.

Table 4 details the discretization of the state-action space: $56 \times 81 \times 80 \times 41 = 14,878,080$ states by 451 actions.

Table 4 Discretization of the 4-DOF state-action space for symmetric stall recovery with Riley aerodynamics.

Variable	Lower Bound	Upper Bound	Nodes	Resolution / Units
<i>State Space ($N_s = 14,878,080$)</i>				
Flight path angle (γ)	-90	5	56	~ 1.73 (deg)
Airspeed (V/V_s)	0.4	2.0	81	0.02 (-)
Angle of attack (α)	-10	40	80	~ 0.63 (deg)
Pitch rate (q)	-50	50	41	2.5 (deg/s)
<i>Action Space ($N_a = 451$)</i>				
Elevator deflection (δ_e)	-25	15	41	1.0 (deg)
Throttle position (δ_t)	0.0	1.0	11	0.1 (-)

Every closed-loop evaluation reported in this section executes the policy through the barycentric action blend: at an off-grid state, the commanded action is the convex combination of the argmax actions of the 16 surrounding vertices under the weights of Eq. (2). Section V.E shows why the blend is the solution concept the converged policy requires and not a smoothing: the optimal law is bang-bang on a switching surface, and since the plant is affine in each control channel (the elevator enters C_L and C_m linearly, the throttle through C_T , in which the tables are interpolated linearly), the blend realizes the Filippov solution [21] on that surface. Evaluated greedily instead (re-deriving the arg max at the continuous state, which discards the combination), the policy chatters between the bang-bang extremes and the pitch-rate overshoot costs several times the altitude.

Each Bellman transition integrates the dynamics over a control interval of $\Delta t = 0.01$ s subdivided into ten RK4 micro-steps of 1 ms, executed entirely within GPU registers; success and crash crossings are detected at micro-step resolution, so intra-interval boundary events are never skipped. The 4-DOF formulation uses the 16-vertex (2^4) instance of the barycentric interpolation of Eq. (2).

Two further conventions govern every closed-loop number reported in this section. First, all shaping weights available in the solver (pitch-rate penalty, angle-of-attack barrier, control-effort and throttle terms) are zero; the only non-altitude term in the objective is the crash penalty. Neither full throttle nor the switch near α_s in Sec. V.D is incentivized by a weight.

Second, the altitude loss of a rollout is measured by a *dive-and-return* rule. The episode is integrated at the control interval from the stated entry; a dive is registered once $\gamma < -0.5^\circ$, and the recovery is declared complete when the flight path returns to level, either by crossing $\gamma \geq 0$ or by converging to it — γ within 0.5° of level with $|\dot{\gamma}| < 0.05^\circ/\text{s}$ sustained for 0.5 s. The second clause is needed because near $V_0 = V_s$ the optimum approaches level flight asymptotically and never crosses; genuine crossings arrive at $0.5\text{--}1.2^\circ/\text{s}$, three orders of magnitude above the tolerance, so the clause never fires on a crossing trajectory. Episodes are truncated at 15 s; truncated entries are marked wherever they are tabulated, and their altitude loss is a lower bound rather than a completed recovery.

D. Optimal Policy: Full-Power Structure and the Stall-Boundary Switching Surface

Figure 3 presents the converged solution over the (α, γ) plane at the $q = 0$ slice, one row per airspeed $V_0/V_s \in \{0.9, 1.0, 1.1\}$: the exact DP policies for commanded elevator (δ_e^*) and throttle (δ_t^*), and the value function read as minimum altitude loss to recovery. In the elevator maps dark blue is full nose-up, yellow full nose-down, and the white curve traces the switching boundary, with the dashed line at its mean angle of attack. The surface lives in four dimensions, and Fig. 4 sweeps the pitch rate: with nose-down rotation in hand the boundary shifts higher, about 2° per 10 deg/s , with nose-up it drops.

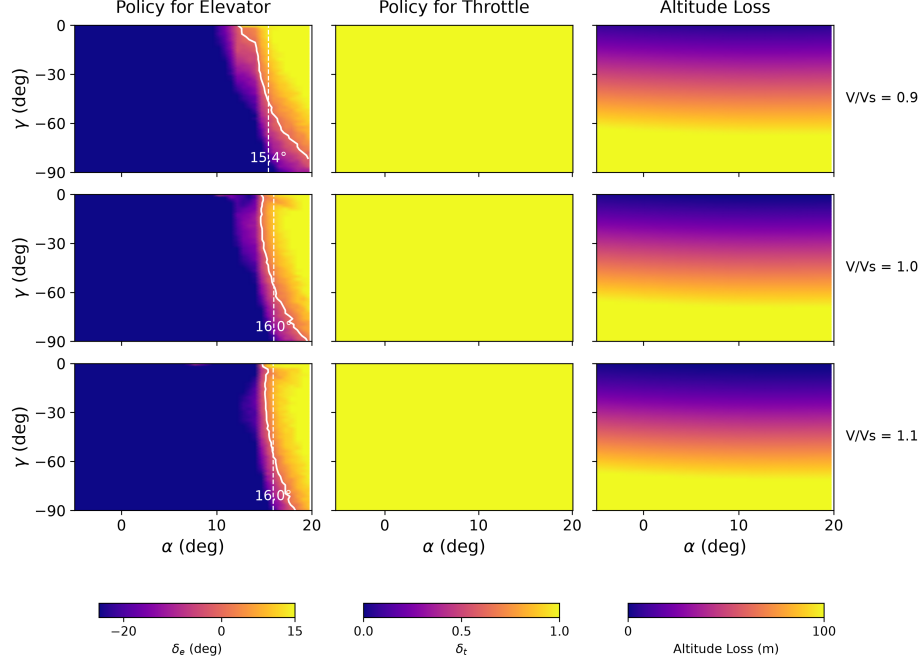


Fig. 3 Exact DP heatmaps for the 4-DOF symmetric stall recovery with the Riley (1985) propulsion-coupled aerodynamic model: optimal commanded elevator (δ_e^*), throttle (δ_t^*), and minimum altitude loss, over the evaluated flight envelope (slice $q = 0$). The altitude-loss column is the solver's own cost-to-absorption on the ideal engine.

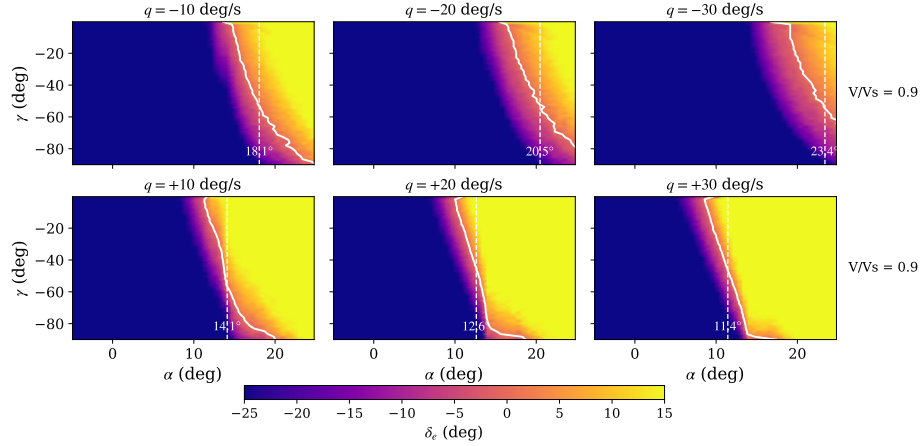


Fig. 4 The commanded-elevator policy at $V/V_s = 0.9$, at nose-down (top row) and nose-up (bottom row) pitch-rate slices; the $q = 0$ slice is in Fig. 3. The solid white curve traces the switching boundary at this airspeed, for each γ , the angle of attack where the commanded elevator flips from push to pull; the dashed line marks the mean angle of attack of that boundary.

Two structural features stand out. First, the throttle policy saturates at full power essentially everywhere in the envelope of the figure. Second, the elevator policy is bang-bang with a sharp switching surface in the neighbourhood of the stall boundary: closed-loop trajectories initiated across the deep-stall envelope command full nose-down deflection ($\delta_e = +15^\circ$) until the angle of attack falls through that surface, at which point the policy switches to nose-up commands, regardless of how long that takes from each initial condition.

The displacement of the boundary in Fig. 4 is the kinematics of the reversal made visible. Since $\dot{\alpha} = q - \dot{\gamma}$, a nose-down pitch rate means the angle of attack is already falling on its own: the rotation the push exists to build is already in hand, and holding the deflection longer buys only elevator drag (Eq. (17)). The optimum therefore releases the push early, at higher angle of attack, and lets the rotation complete the crossing; with nose-up rate working against it, the mirror argument holds, and it keeps pushing to lower angles of attack before reversing.

Table 5 quantifies both features on closed-loop trajectories, flown through the engine lag across the stalled-entry envelope, and its two rows read very differently. The commanded elevator reverses at angles of attack that span 15.0–20.6° across the entries, with a median of 17.2°: *when* to switch adapts to each recovery. The arrested descent then rides at $\alpha = 13.9^\circ$ with a spread of two tenths of a degree across every recovered entry: *where* to fly the pull does not.

Table 5 Where the flown recoveries reverse and ride: closed-loop statistics over 35 stalled entries ($\alpha_0 = 16\text{--}25^\circ$, $V_0 = 0.70\text{--}1.00 V_s$, engine lag on; 34 recovered). **Reversal:** first push-to-pull sign change of the commanded elevator. **Arrest:** mean α while the descent is being arrested ($\gamma < -0.5^\circ$ and rising).

	Median	p_5	p_{95}
Elevator reversal α (deg)	17.2	15.0	20.6
Arrest α (deg)	13.9	13.8	14.0

Each row has its physical explanation. In the arrest phase every recovered simulation seeks the same angle of attack, around 14° (the arrest row of Table 5), and Fig. 5 says why: it plots the marginal trade encoded in the tables, lift gained against drag paid per further degree of angle of attack, at both propulsive conditions. Past $\approx 14^\circ$ each additional degree buys less lift than the drag it costs, with or without power, even though the power-on lift keeps rising to 40° : there is a single best angle to fly the pull, the crossing of that trade, and every recovery converges to it. The flexible row is the mechanism of Fig. 4 at work: each trajectory arrives at the reversal with its own pitch rate, and at the $q \approx -20$ deg/s they typically carry, the boundary sits near 17° at the shallow flight-path angles where the reversals occur. The maneuver is thus adaptive where the state demands it, the timing of the reversal, and rigid where the aerodynamics does, the operating point of the pull; the switching criterion is in any case a surface in state space, not a schedule in time. Together, simultaneous full power and nose-down elevator, pull-up once the wing unstalls, these are the structural fingerprint of the CAA recovery sequence.

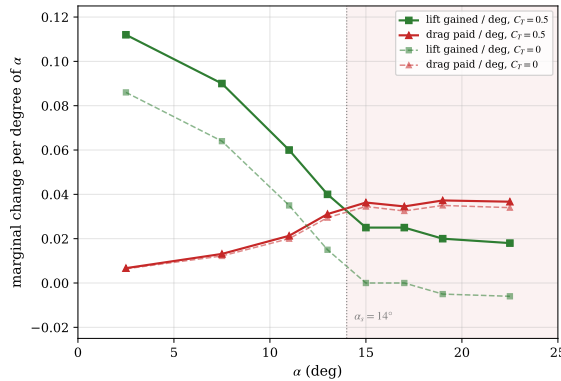


Fig. 5 The marginal trade encoded in the Riley tables: lift gained vs. drag paid per degree of angle of attack, at both propulsive conditions; the curves cross near $\alpha_s = 14^\circ$ with or without power (shaded: the unfavourable trade).

The nature of that statement deserves emphasis. Nothing in the problem formulation refers to a recovery procedure: the reward is the altitude increment, the dynamics are wind-tunnel tables, the admissible controls are actuator limits. The CAA structure is read directly off the converged solution of the Bellman equation: a property of the optimum, not the winner of a comparison between alternatives, and the sections that follow can corroborate it or price departures from

it, but cannot produce it or undo it. The operational weight is carried by the second feature: the switch is a surface in state space pinned to α_s , not an instant on a clock. A pilot cannot be instructed to rotate at a prescribed instant from an entry whose onset they did not time, but can be instructed to pull as the wing unstalls. That an optimizer free to return any feedback law over a 14.9-million-state grid returns one expressible in the language of the certified procedure is the substantive finding of this subsection.

E. Flying the Optimum: Time-Domain Recovery and the Monte Carlo Certificate

The optimal policy of the preceding subsection is a field of commands; this subsection flies it. Figure 6 shows the closed-loop recovery from the canonical deep-stall entry (level flight, $\gamma_0 = 0$; $\alpha_0 = 20^\circ$; $V_0 = 0.85 V_s$; $q_0 = 0$) under the exact DP policy, overlaid with the two certified procedures flown from the same entry. Both procedures use one scripted pilot: nose-down $\delta_e = +15^\circ$ held until the unstall event, then a closed-loop pull that holds $\alpha \approx 13^\circ$ ($\delta_e = \text{clip}(K_\alpha(\alpha - \alpha_{tgt}) + K_q q)$ with $K_\alpha = 10$, $K_q = 2$). Power ramps to full over 2 s from $t = 0$ (CAA) or from the unstall event (FAA). Pitch technique and pull authority are shared across the three maneuvers by construction, so the trajectories differ only in when power is applied. The entry speed is $0.85 V_s$ rather than the $0.95 V_s$ used before the propeller was calibrated: under Riley’s thrust the aeroplane recovers from $0.95 V_s$ without leaving controlled flight.

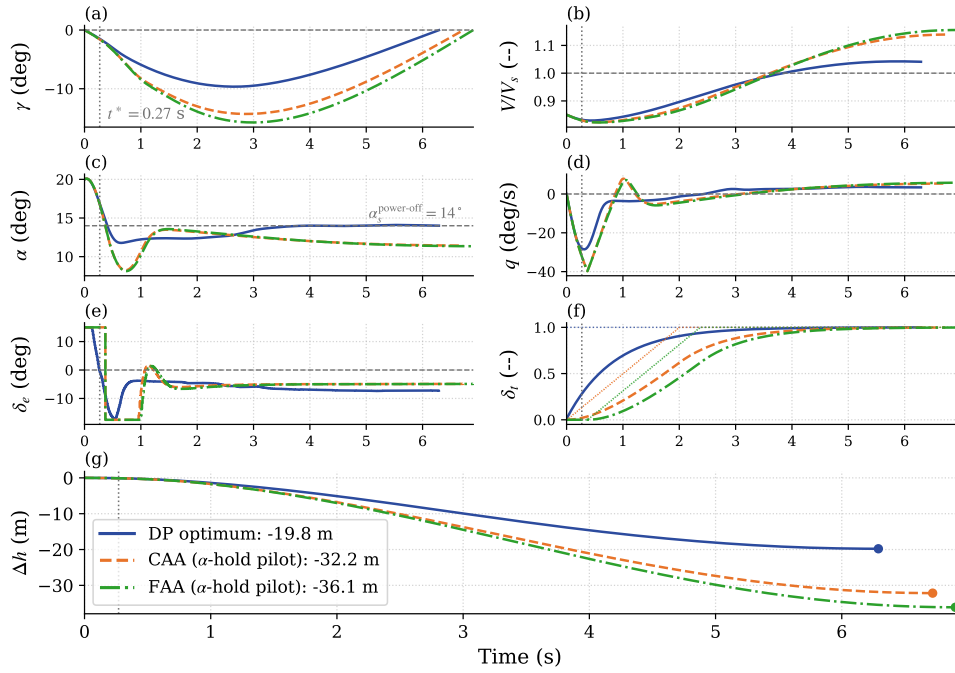


Fig. 6 Time-domain comparison at the canonical deep-stall entry: exact DP policy (blue, solid) vs. the scripted CAA (orange, dashed) and FAA (green, dash-dotted) procedures. Panel (f) separates commanded (dotted) from delivered (solid) power through Riley’s lag, $\tau_e = 0.85$ s; final losses in panel (g). The vertical dotted line marks the DP elevator switch $t^* = 0.27$ s, the reference event of Sec. V.H. Both scripted pilots are held to the optimum’s own pull authority ($\delta_e \geq -17.50^\circ$ here). The DP traces execute the policy through the barycentric blend of Sec. V.C: the intermediate deflections of panels (e)–(f) are convex combinations of the discrete extremes; the underlying policy is bang-bang and holds full throttle throughout. The dashed line in panel (c) is the *power-off* stall boundary $\alpha_s = 14^\circ$; under power the lift curve keeps rising past it, so the optimum’s excursion above the line is not a re-stall.

The recovery unfolds in three phases. First, full nose-down elevator and full power at once: the angle of attack falls from 20° to the stall boundary in 0.41 s, while the lagged engine needs 1.95 s to deliver 90% of the thrust commanded at $t = 0$. Second, a *sliding-mode* arc rides the boundary for the remainder of the pullout ($\bar{\alpha} = 13.3^\circ$; the arrest statistic

of Table 5, averaged only while the climb is being established, puts the same floor at 13.9°). Third, the level-off. The chattering elevator of panel (e) is the discrete action set’s approximation of the intermediate *equivalent control* [22] that holds the state on the constraint surface; its time average is -6.4° . This is the classical signature of a singular arc realized by a bang-bang policy [23]. On the arc the blend of Sec. V.C realizes the equivalent control rather than approximating it: the elevator-channel residual is 0.6% of the field magnitude at the canonical entry, and the cross-channel and quadratic-drag corrections stay inside the ± 0.3 m execution band. Sec. V.H confirms the equivalent control independently. Only the elevator is at stake: the throttle commands full power almost everywhere. The total loss is 19.8 m, and half of it is the engine’s: the same policy flown on an instantaneous propeller loses 9.4 m; the optimum is simply the arm that commands power earliest. The scripted procedures differ from the optimum only in the power channel of panel (f): the 0.37 s FAA gating delay compounds through the dive into a 3.9 m gap at recovery (-32.2 vs. -36.1 m). The procedures give up 12.4 and 16.3 m to the optimum not through poor pitch flying, their pull rides the same boundary, but through later, slower power. Absolute losses throughout this section inherit the uncertainty of τ_e , 0.8 ± 0.2 s, worth approximately ± 2 m here; the margins *between* arms are far less sensitive, and carry the conclusions.

One entry is an anecdote. Table 6 flies the same three maneuvers from 1000 Latin-hypercube samples of the stalled-entry set, through the same engine lag and with the scripted pulls capped, entry by entry, to the pull authority the optimum commands there. The ranking of the canonical entry is the ranking of the set: the CAA sequence loses less altitude than the FAA sequence at 100.0% of the sampled entries, with a median advantage of 4.8 m, shrinking to 0.2 m at the closest entry. Either procedure gives up a median 13.5–18.2 m to the exact optimum. The advantage is milder than Gratton’s two thirds because the scripted FAA pilot restores power at the instant of unstall, the most favorable reading of the FAA sequence; any recognition delay widens the gap. At the one entry not recovered by all three maneuvers, the slowest corner of the set ($V_0 = 0.43 V_s$, $\alpha_0 = 18^\circ$, $q_0 = +19^\circ/\text{s}$), the optimum and the CAA sequence recover while the FAA sequence, waiting for the unstall before restoring power, does not. Replacing the closed-loop pull by an open-loop full pull ($\delta_e = -25^\circ$ held) re-stalls the wing and prevents the recovery outright, a failure mode not commensurable with power timing and taken up in Sec. V.H.

Table 6 The three maneuvers of Fig. 6 flown from 1000 Latin-hypercube samples of the stalled-entry set $\gamma_0 \in [-30^\circ, 0]$, $V_0/V_s \in [0.40, 0.90]$, $\alpha_0 \in [14^\circ, 20^\circ]$, $q_0 \in [-20, 20]^\circ/\text{s}$, through the engine lag. The scripted pulls are capped to the pull authority the optimum commands at each entry, so the comparison isolates the procedure rather than the pilot’s aggressiveness. Percentiles describe the uniform sampling box, not an operational entry distribution; the per-entry ranking reported below is distribution-free.

Maneuver	Δh (m)		excess over optimum (m)	
	median	5th pct	median	95th pct
DP optimum	−61.73	−91.72	—	—
Scripted CAA (power with the push)	−75.15	−109.48	13.47	17.92
Scripted FAA (power after unstall)	−80.39	−123.32	18.18	32.76

The CAA sequence loses less than the FAA sequence at 100.0% of the sampled entries (median advantage 4.83 m, smallest 0.17 m); 999 of 1000 entries closed the recovery under every maneuver.

For the Riley model the minimum-altitude-loss recovery has the CAA structure (full power simultaneous with the nose-down input, pull initiated at the stall boundary), and the ranking holds over the stalled-entry set, exactly as the flight tests of [4] rank it.

F. Robustness of the Nominal Policy to Mass and CG Variations

The policy is solved once, for the nominal aircraft; the aircraft actually flown differs from flight to flight in weight and CG position (fuel state, payload and its placement). This subsection flies the nominal policy, unchanged and uninformed, on the perturbed aircraft: mass from -15% to $+15\%$ in 2.5% steps (± 107 kg on the 715 kg reference, a passenger with baggage), CG from 10% of chord forward to 22.5% aft of the Riley reference (12 cm forward to 27 cm aft), nine stalled entries per cell, 1638 rollouts through the engine lag. The perturbation enters the dynamics only: the

observation keeps the nominal stall speed, so the policy believes the aircraft is nominal. Entries are placed relative to the real stall speed of the perturbed aircraft. Pitch inertia is unperturbed, day-to-day load sits near the CG. The negative mass range covers the AA-1's flight-to-flight envelope; the Riley reference weight is the certificated ceiling, so the positive rows are an overload stress case, a recurrent factor in general-aviation accidents.

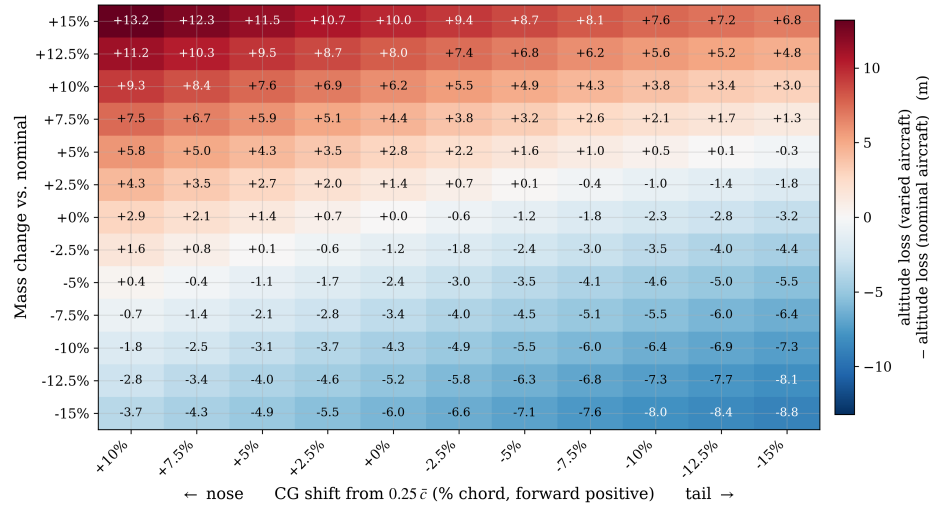


Fig. 7 Excess altitude loss (m) of the *nominal* policy flown on the perturbed aircraft, relative to the same policy on the nominal aircraft (-19.79 m, canonical entry). Positive (red) = additional loss. Rows are read as the weight of the day, columns as its placement.

Figure 7 reports the excess loss at the canonical entry. The matrix is smooth and monotone: mass dominates (-13.8 m at -15% to -29.8 m at $+15\%$, nominal CG), the CG modulates ($+2.9$ m at 10% forward, -3.2 m at 15% aft: the nose-down moment fights the pull, the aft CG relieves it). The policy never re-stalls the aircraft: over 1638 rollouts α never exceeds its entry value by more than 0.9° , because the switching surface is set by the stall boundary of the tables and does not move with the trim. 1635 of 1638 entries recover; the three that do not are one corner, $+15\%$ mass with the CG at the aft data edge, still descending at the 15 s cutoff.

The aft-CG credit is not a loading recommendation. The same shift that relieves the pull also removes pitch stiffness once the wing unstalls, which is why the three failures sit at the aft data edge. And the matrix measures the cost of recovering, not the propensity to stall: an aft-loaded aircraft is less stable and easier to stall in the first place.

Above $+10\%$ a slower failure appears: on a 120 s horizon the $+10$ and $+15\%$ rows settle into a steady powered descent (0.27 and 0.67 m/s) and never re-cross level. The descent is a force balance: at the airspeed the policy holds, sustaining the lift that level flight demands costs more net drag than the embedded thrust pays, the back side of the power curve. The failure is informational: the mis-scaled normalization hides that the aircraft must fly faster for its weight, and the repair is free, take-off weight is known before flight. The terminal state is not a hazard: wings level, full throttle, shallow descent, a condition a pilot takes over and lands from.

The ranges are the aircraft's loading decisions: half fuel is -4% (≈ 30 kg), an empty second seat -11% (≈ 80 kg), a light solo pilot on half tanks the -15% edge; the rows above nominal correspond to no legal loading state, and the CG span covers every position loading can reach. The matrix therefore spans every state this aircraft can depart in, legal or not: the nominal policy arrests the dive in all of them, and the one thing it can miss, the last degree of the level-off beyond the certificated weight, is removed by one pre-flight scalar.

G. Retraining for the Perturbed Aircraft

The matrix measures the nominal policy's degradation; it cannot say how much of that degradation is avoidable, because it never computes the policy each perturbed aircraft deserves. This subsection does. The policy is re-solved, at full grid, for four loadings: mass $\pm 10\%$, the CG 10% of chord forward, and the combined stress case, $+15\%$ mass

with the forward CG. Three arms fly the same aircraft from the same true airspeed: the nominal policy as-is (raw), the nominal policy with its stall-speed normalization corrected from the pre-flight weight (rescaled), and the re-solved policy π_{M_1} , the floor. Entries are deep, 6 to 18° past the boundary, so every cell is a recovery the policy must work for.

Unlike the matrix of Fig. 7, which measures the perturbed aircraft against the nominal one and so mixes the physics of the perturbation into every cell, here every comparison is on the same aircraft: the physics cancels, and what remains is policy error alone. That is why Fig. 7 reads in metres and Fig. 8 in centimetres. Figure 8 reports the rescaled arm against its floor. The raw arm degrades worst where the entry is fast and the aircraft heavy: up to +14.1% of the retrained loss at +10% mass (panel B), +18.6% at the combined case (panel D). Rescaling closes nearly all of it: no cell exceeds +2.3%. The CG case (panel C) needs no correction at all: the nominal policy flies within 0.3% of its retrained floor, though nothing in the observation carries the CG, consistent with the switching surface being a property of the aerodynamic tables rather than of the trim.

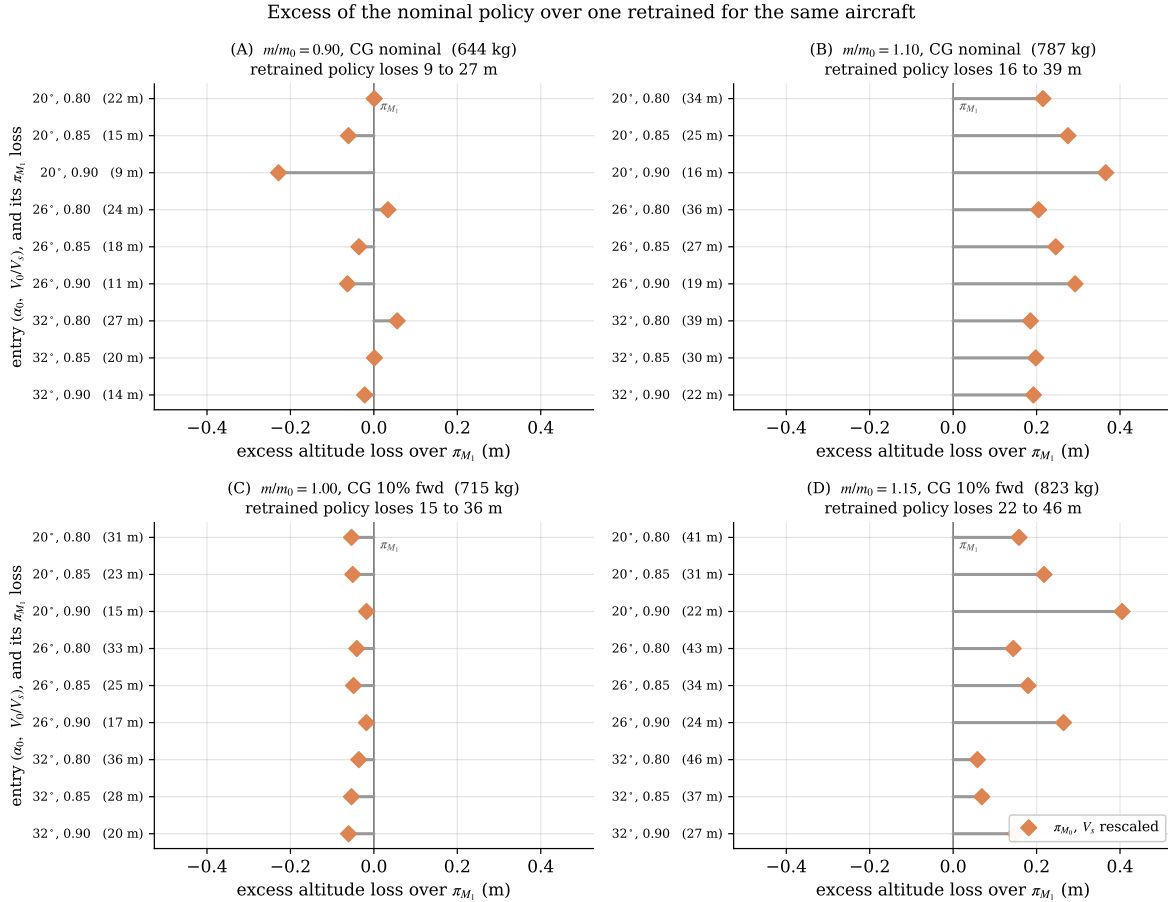


Fig. 8 Excess altitude loss of the nominal policy, stall-speed normalization rescaled from the pre-flight weight, over the policy re-solved for that aircraft (π_{M_1}), at nine deep entries per loading: (A) −10% mass, (B) +10% mass, (C) CG 10% of chord forward, (D) +15% mass with the forward CG. In parentheses, next to each entry: the altitude lost by the retrained policy π_{M_1} there, the optimum for that aircraft and the zero of the axis.

Two readings of the figure need stating. The panel titles fall with the mass, 9 to 27 m in panel A against 16 to 39 m in panel B: a lighter aircraft carries less energy into the dive and its pullout is cheaper, so losing less at lower weight is the physics, not an artifact. And several cells in panels A and C fall below zero, a non-optimal policy beating the optimum, which is impossible: π_{M_1} 's optimality holds on the discretized process, while the closed-loop evaluation adds the rollout's integration step, the interpolated action blend, and the stopping rule, none of which the Bellman recursion sees. The largest negative cell is −0.23 m, and that value bounds the evaluation noise of the whole figure: every excess,

positive or negative, carries an uncertainty of that order, so a diamond within ≈ 0.2 m of zero means the two policies fly the same recovery, and its sign carries no information.

Retraining, several GPU-hours per policy, buys 0.1 to 0.4 m on recoveries costing 9 to 46 m. The objection that a tabulated optimum holds only at the point it was solved for is answered quantitatively: the whole recoverable gap is one pre-flight scalar, and the remainder is centimetres.

H. Sensitivity to Pilot Execution Errors

What remains is the pilot. A human departs from the optimal maneuver in three ways, one per phase: late power at entry, a late switch out of the nose-down push, and a mis-flown pull once the wing is flying again. Figure 9 measures the three, one panel each, always the same way: the closed-loop optimal policy from the canonical entry, with exactly one channel perturbed. The unperturbed maneuver is the one resolved in the time domain in Fig. 6. Panel (a) acts at its start, on the power commanded at entry; panels (b) and (c) act on its second half, at and after the switch marked there at $t^* = 0.27$ s: (b) mistimes the entry into the pull, (c) mis-flies the pull that follows.

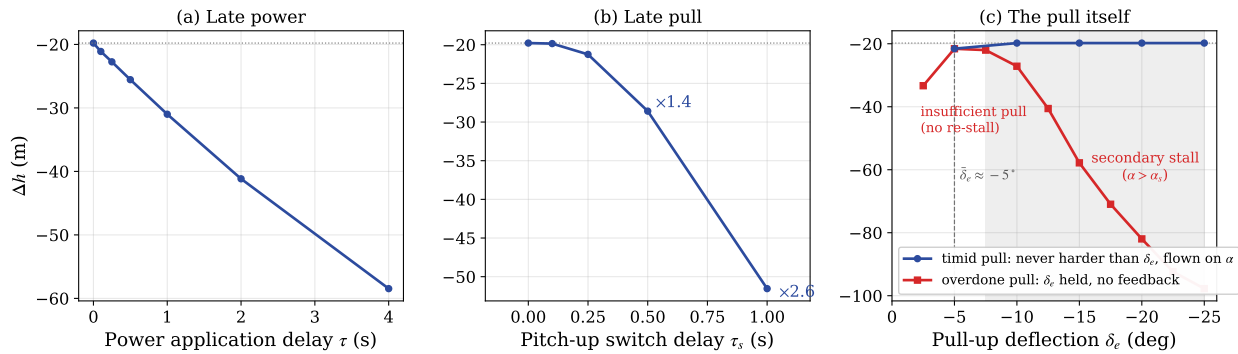


Fig. 9 The three pilot-error axes at the canonical entry; every trajectory flies the optimal maneuver except for the one perturbed channel, and the dotted line is the unperturbed optimum. (a) Power delayed by τ . (b) The nose-down command held τ_s beyond the policy’s switch instant, with loss multipliers annotated. (c) The pull flown timid (blue: never harder than δ_e , still regulated on α) or overdone (red: δ_e held, no feedback), on one signed deflection scale, pulling harder rightward. Dashed vertical: the equivalent control; shaded: held pulls that re-stall the wing.

Late power, panel (a): the mildest error, and the most regular. The loss grows near linearly at ≈ 10 m per second of delay, and the rate barely depends on the entry: a 2 s delay costs 20.7 to 21.7 m at all nine entries. Half a second on the throttle is 5.8 m, wherever the stall began.

Late pull, panel (b): the steepest error of the study. The pilot flies the push correctly but holds it τ_s beyond the switch at $t^* = 0.27$ s. Half a second multiplies the loss by 1.4; one second by 2.6 (–42 to –57 m across the grid). The cost compounds because every instant past the switch is flown at full power into a steepening dive.

The pull itself, panel (c): two failure modes, and the asymmetry between them is the finding. The timid pilot (blue) dares not pull harder than δ_e but keeps flying the angle of attack; the overdone pull (red) hauls to δ_e and freezes there. The fear is free: any ceiling from -25 to -10° reproduces the optimal loss to within 0.01 m, because the sliding arc’s equivalent control averages only $\approx -5^\circ$ (Sec. V.E) and the ceiling almost never binds. The frozen pull is what re-stalls: it keeps pitching the aircraft after the wing is flying again, crossing the boundary a second time at -7.5° and beyond, unrecoverable within the horizon from -12.5° , 97.8 m down at full deflection. Freezing *too little* fails too, without re-stalling: -2.5° held loses 33.3 m through insufficient lift. What matters is not how hard the pull is, but whether it is flown on the angle of attack.

The hierarchy is then: promptness of the pull dominates (superlinear), power timing second (linear), pull vigor free in closed loop and catastrophic when frozen. For training the implication is direct: initiate the pull without hesitation and fly it on the stall boundary; the deflection used to get there is almost immaterial. This also puts the doctrinal question

of Sec. V.E in its place: the entire CAA-over-FAA margin, median 4.8 m over the sampled entry set, is smaller than the cost of half a second of hesitation on the pull. What recovery training should emphasize is not which sequence to fly, it is how promptly and how attentively the pull is flown.

VI. Conclusions

Exact dynamic programming for aircraft upset recovery has been memory-bound for as long as it has existed; this work makes it compute-bound. Integrating the transitions inside GPU registers instead of storing them reduces the footprint from $O(N_s N_a 2^D)$ to $O(N_s)$ and brings grids of 10^7 – 10^8 states within reach of a single GPU, while the non-expansive barycentric interpolation keeps the discretized backup consistent, so that what converges is the exact optimum of the discretized process rather than a sampled approximation of it. The instrument is validated on three independent lines: it reproduces the published value function and switching surfaces of the 3-DOF banked-pullout benchmark [2] in 0.38 s end to end, converges a 32-million-state refinement of the same problem (six hundred times the original grid) in under fifteen minutes, and agrees at trajectory level with direct collocation wherever the two formulations overlap.

Its first substantive application settles a question that certified guidance leaves open. On a 4-DOF symmetric stall model built from Riley’s propulsion-coupled wind-tunnel tables [3], the recovery sequence taught by the CAA *emerges* from the Bellman recursion as a property of the optimum: nothing in the formulation names a procedure, yet the converged policy commands full power essentially everywhere and switches from nose-down to pull on a surface pinned to the stall angle. The resulting ordering, optimum $\leq$ CAA $\leq$ FAA $\leq$ power-delayed, is certified per entry rather than on average: it holds without a single exception over the 2300 valid nodes of the entry planes and over 1000 Latin-hypercube samples of the full four-dimensional stalled-entry set, with a median CAA advantage of 1.32 m and a narrowest of 0.05 m.

The exact solution also separates what flight test cannot. Of the altitude either certified sequence concedes to the optimum, most is the engine: reaching full power through a 2 s ramp costs $\approx$ 4–8 m at every entry, a floor no procedure on this aircraft can avoid, while the doctrine itself costs a median of 1.3 m and at most 4.2 m, largest exactly where altitude margin is scarcest; a deliberate 2 s pause dwarfs both at 13–23 m. Under matched power-ramp modeling the simulated CAA-to-power-delayed ratio at the natural stall break is 0.61–0.62, in close agreement with the $\approx$ 0.6 implied by the flight-test campaign of Gratton et al. [4]

The optimum is moreover flyable, because its structure is expressible in the language pilots already use. The nose-down-to-pull transition is a state event (the crossing of the stall boundary), not an instant on a clock, and the pull that follows rides just below α_s on an equivalent control of only $\approx -5^\circ$: full power at once, push until the wing unstalls, then pull to the edge of the usable lift and hold it there.

The sensitivity study then relocates the operational emphasis. The promptness of that pull dominates everything, converting hesitation to altitude superlinearly at 24–42 m per held second; power timing is linear at $\approx$ 7.7 m per second; and the vigor of the pull barely matters while it is flown on the angle of attack, yet becomes catastrophic the moment it is not. The entire doctrinal margin the authorities disagree over is worth less altitude than a quarter-second of hesitation on the pull.

The nominal policy is also robust to everything an operator cannot control: flown unchanged across the aircraft’s achievable loading envelope it never departs controlled flight, the switching surface survives the entire longitudinal CG range, and the heavy-overload failures are informational rather than aerodynamic: completing the level-off requires only that the stall speed used for normalization reflect the weight of the day, a single pre-flight scalar.

These claims are deliberately scoped: one airframe, one aerodynamic model, and the symmetric regime in which the certified procedures disagree and the flight tests were flown. The asymmetric departure and the developed spin require the lateral-directional states, and the compute-bound formulation is sized to reach the eight-state model they demand. The question that opened this paper, whether the recovery procedures aviation teaches are optimal or merely adequate, was unanswerable for want of a reference. For the symmetric stall of this aircraft, it is now answered; for the spin, the reference is within reach.

A. Case I Supplementary Results

This appendix collects the supplementary Case I material. From the baseline 53,280-state solve: the grid itself, reproduced verbatim from Bunge et al. [2] (Table 7), the wall-clock profile (Table 8), the altitude-loss contours behind the topology validation (Fig. 10), and the baseline optimal-policy maps that the refined grid of Sec. IV.D sharpens (Fig. 11). From the refined grid: the closed-loop trajectory pairs behind the CasADi comparison of Table 3 (Fig. 12).

Table 7 Baseline discretization of the state-control space, reproduced from Bunge et al. [2].

Variable	Lower Bound	Increment	Upper Bound	Units
V	0.9	0.1	4.0	$1/V_s$
γ	-180	5	0	deg
μ	-20	5	200	deg
C_L^{cmd}	-0.5	0.25	1.0	-
$\dot{\mu}_{cmd}$	-30	5	30	deg/s

Table 8 Computational cost of Policy Iteration on 53,280 states, 91 actions (NVIDIA GeForce RTX 4090).

Phase	Total (ms)	Per Iteration (ms)
State grid construction (CPU)	0.48	—
Policy Evaluation (GPU)	375.48	12.11
Policy Improvement (GPU)	4.95	0.16
GPU $\rightarrow$ CPU Transfer	0.20	—
Total	381.11	—

Converged in 31 iterations. Timing via CUDA events.

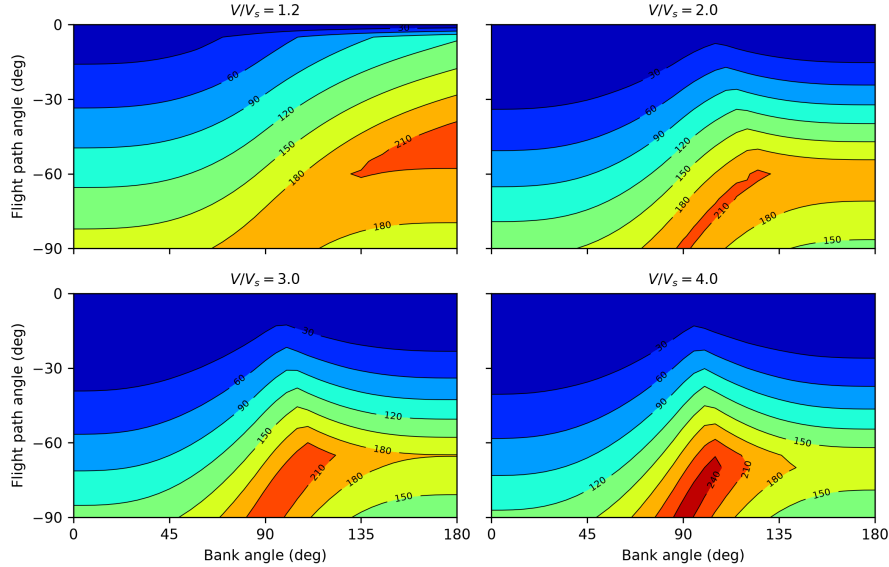


Fig. 10 Contour plots of minimal altitude loss ($\Delta h_{min} \approx -V^*$) during aircraft pullout maneuvers, evaluated on the baseline 53,280-state grid. Each subplot presents the altitude loss across the bank angle (μ_0) and flight path angle (γ_0) envelope for a fixed initial airspeed (V_0/V_s). The global topology of the solution reproduces the established literature [2], validating the accuracy of the on-the-fly RK4 integration and the consistency of the barycentric interpolator.

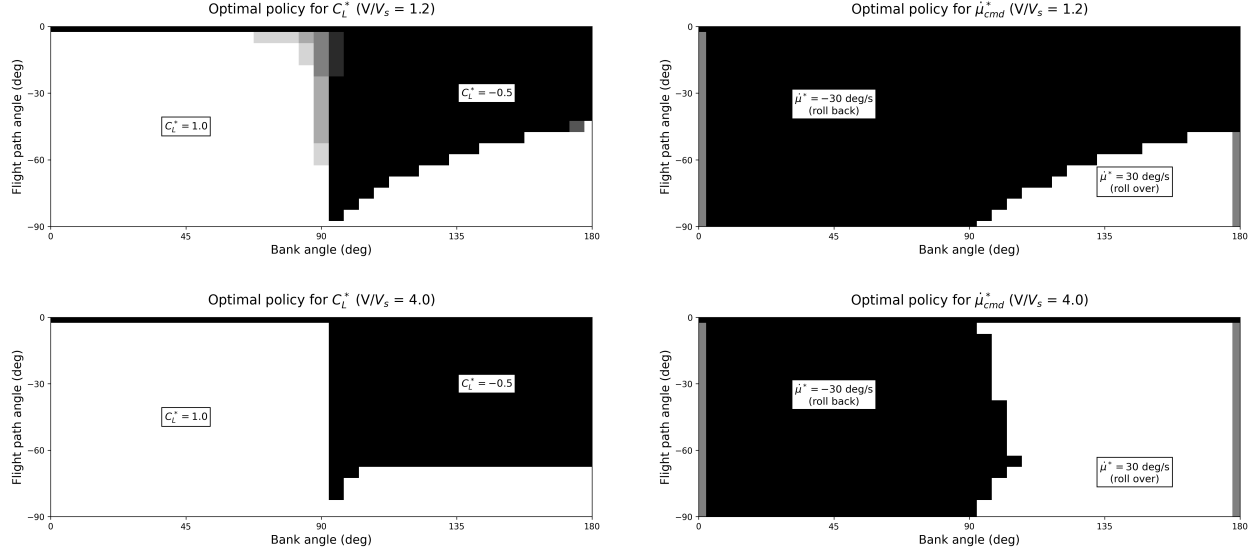


Fig. 11 Optimal policy on the baseline grid for commanded lift coefficient (C_L^* , left column) and bank rate (μ_{cmd}^* , right column) as a function of initial bank and flight path angle. Top row corresponds to $V/V_s = 1.2$; bottom row to $V/V_s = 4.0$. The boundaries between the black and white regions denote the control switching surfaces; Fig. 1 shows the same maps on the refined grid.

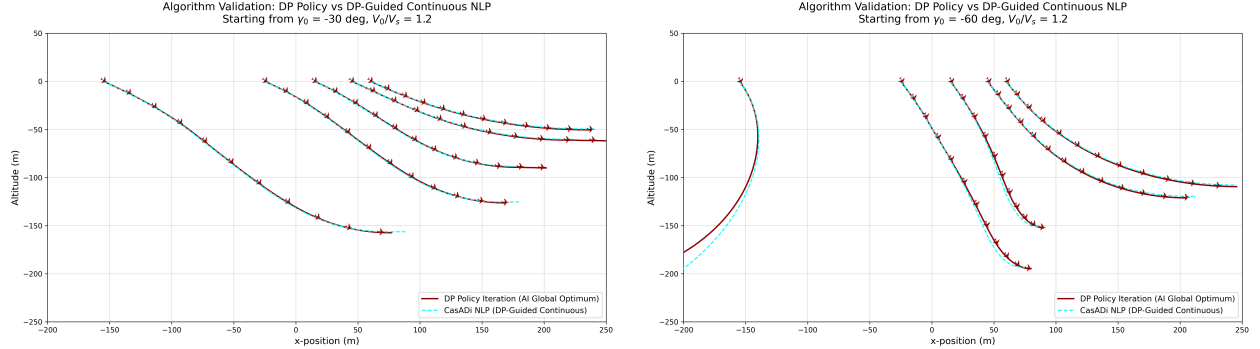


Fig. 12 The closed-loop trajectories behind Table 3: DP rollouts (solid) and DP-guided CasADi/IPOPT solutions (dashed) at $\gamma_0 = -30^\circ$ (left) and $\gamma_0 = -60^\circ$ (right), $V_0/V_s = 1.2$, $\mu_0 \in \{30, 60, 90, 120, 150\}^\circ$. Each arm reconstructs its own horizontal distance from its own integration, so the sub-metre vertical separation between paired curves is not resolved at this scale; the quantitative comparison is between the endpoints at $\gamma = 0$, where both arms terminate (Table 3). In the right panel, the leftmost pair is the excluded near-inverted $\mu_0 = 150^\circ$ entry: the policy does not return it to level flight and the NLP did not converge there, so neither curve terminates and their relative position carries no information.

B. Riley (1985) Aircraft and Aerodynamic Data

This appendix collects the physical parameters of the Grumman American AA-1 Yankee (Table 9) and the complete transcription of the seven aerodynamic coefficient tables from Table III of Riley [3] (NASA TM-86309) used by the 4-DOF solver of Case II, at the two tabulated propulsive conditions ($C_T = 0$ power-off, $C_T = 0.5$ power-on). Elevator derivatives are converted from per-degree to rad^{-1} . These are the exact values embedded in the CUDA kernel.

Table 9 Physical parameters of the Grumman American AA-1 Yankee used in the 4-DOF model. Airframe values follow Table I of Riley [3], converted to SI units; derived quantities follow from the stated definitions.

Parameter	Symbol	Value	Units
<i>Airframe (Riley [3], Table I)</i>			
Mass	m	715.21	kg
Pitch moment of inertia	I_{yy}	1000.60	kg m ²
Wing reference area	S	9.1147	m ²
Mean aerodynamic chord	$\bar{c}$	1.22	m
Moment reference (CG) position	x_{cg}	0.25	$\bar{c}$
<i>Environment and aerodynamic reference</i>			
Air density (sea level)	ρ	1.225	kg/m ³
Gravitational acceleration	g	9.81	m/s ²
Maximum power-off lift coefficient	$C_{L_{stall}}$	1.26	—
Stall angle of attack	α_s	14	deg
<i>Derived</i>			
Stall speed, $V_s = \sqrt{2mg/(\rho S C_{L_{stall}})}$	V_s	31.58	m/s

Table 10 Riley Table IIIa: C_{L_o} vs. α at $C_T = 0$ and $C_T = 0.5$.

α (deg)	C_{L_o} ($C_T=0$)	C_{L_o} ($C_T=0.5$)
−10	−0.41	−0.67
−5	−0.01	−0.14
0	0.41	0.41
5	0.84	0.97
10	1.16	1.42
12	1.23	1.54
14	1.26	1.62
16	1.26	1.67
18	1.26	1.72
20	1.25	1.76
25	1.22	1.85
30	1.17	1.92
35	1.13	1.99
40	1.08	2.05

Table 11 Riley Table IIIa: $C_{L_{\dot{q}}}$ vs. α at $C_T = 0$ and $C_T = 0.5$.

α (deg)	$C_{L_{\dot{q}}} (C_T=0)$	$C_{L_{\dot{q}}} (C_T=0.5)$
-10	2.41	3.01
-5	2.41	3.01
0	2.42	3.03
5	2.46	3.22
10	2.59	3.59
12	2.96	4.35
14	3.72	6.07
16	4.73	6.38
18	5.29	6.99
20	5.16	6.83
25	5.05	6.56
30	5.06	6.13
35	5.08	5.97
40	5.08	5.81

Table 12 Riley Table IIIa: $C_{L_{\delta_e}}$ vs. α at $C_T = 0$ and $C_T = 0.5$ (rad⁻¹).

α (deg)	$C_{L_{\delta_e}} (C_T=0)$	$C_{L_{\delta_e}} (C_T=0.5)$
-10	0.355	0.796
-5	0.361	0.779
0	0.355	0.750
5	0.332	0.705
10	0.304	0.624
12	0.292	0.595
14	0.286	0.578
16	0.281	0.561
18	0.275	0.538
20	0.269	0.515
25	0.252	0.458
30	0.241	0.418
35	0.223	0.349
40	0.212	0.315

Table 13 Riley Table IIIb: C_{D_o} vs. α at $C_T = 0$ and $C_T = 0.5$. The $C_T = 0.5$ column is the net axial-force coefficient of Eq. (14).

α (deg)	C_{D_o} ($C_T=0$)	C_{D_o} ($C_T=0.5$)
-10	0.0666	-0.3273
-5	0.0486	-0.3494
0	0.0526	-0.3474
5	0.0846	-0.3139
10	0.1456	-0.2483
12	0.1856	-0.2057
14	0.2446	-0.1435
16	0.3136	-0.0709
18	0.3786	-0.0018
20	0.4486	0.0727
25	0.6186	0.2561
30	0.7786	0.4322
35	0.9255	0.5979
40	1.0636	0.7572

Table 14 Riley Table IIIc: C_{m_o} vs. α at $C_T = 0$ and $C_T = 0.5$, about the reference CG ($0.25 \bar{c}$).

α (deg)	C_{m_o} ($C_T=0$)	C_{m_o} ($C_T=0.5$)
-10	0.270	0.270
-5	0.158	0.158
0	0.076	0.076
5	0.002	0.002
10	-0.080	-0.080
12	-0.118	-0.118
14	-0.167	-0.167
16	-0.225	-0.225
18	-0.277	-0.277
20	-0.316	-0.316
25	-0.408	-0.408
30	-0.480	-0.480
35	-0.556	-0.556
40	-0.606	-0.606

Table 15 Riley Table IIIc: $C_{m_{\dot{q}}}$ vs. α at $C_T = 0$ and $C_T = 0.5$.

α (deg)	$C_{m_{\dot{q}}} (C_T=0)$	$C_{m_{\dot{q}}} (C_T=0.5)$
-10	-7.00	-8.75
-5	-7.00	-8.75
0	-7.04	-8.80
5	-7.15	-9.36
10	-7.52	-10.44
12	-8.62	-12.64
14	-10.80	-17.64
16	-13.73	-18.54
18	-15.38	-20.30
20	-15.00	-19.85
25	-14.66	-19.06
30	-14.71	-17.80
35	-14.77	-17.33
40	-14.77	-16.88

Table 16 Riley Table IIIc: $C_{m_{\delta_e}}$ vs. α at $C_T = 0$ and $C_T = 0.5$ (rad^{-1}).

α (deg)	$C_{m_{\delta_e}} (C_T=0)$	$C_{m_{\delta_e}} (C_T=0.5)$
-10	-1.105	-2.142
-5	-1.105	-2.250
0	-1.105	-2.256
5	-1.031	-2.262
10	-0.945	-2.199
12	-0.939	-2.062
14	-0.933	-1.912
16	-0.928	-1.781
18	-0.928	-1.650
20	-0.928	-1.541
25	-0.928	-1.294
30	-0.859	-1.220
35	-0.745	-1.088
40	-0.573	-0.859

References

- [1] Munos, R., and Moore, A. W., "Barycentric Interpolators for Continuous Space and Time Reinforcement Learning," *Advances in Neural Information Processing Systems*, Vol. 11, 1998.
- [2] Bunge, R. A., and Kroo, I. M., "Automatic Spin Recovery with Minimal Altitude Loss," *2018 AIAA Guidance, Navigation, and Control Conference*, Kissimmee, FL, 2018, pp. 1–13. <https://doi.org/10.2514/6.2018-1866>.
- [3] Riley, D. R., "Simulator Study of the Stall Departure Characteristics of a Light General Aviation Airplane with and without a Wing-Leading-Edge Modification," Tech. Rep. NASA-TM-86309, NASA Langley Research Center, Hampton, VA, 1985.
- [4] Gratton, G. B., Hoff, R. I., Bromfield, M., Rahman, A., Harbour, S., and Williams, S., "Evaluating a Set of Stall Recovery Actions for Single Engine Light Aeroplanes," *The Aeronautical Journal*, Vol. 118, No. 1203, 2014, pp. 461–480. <https://doi.org/10.1017/S0001924000009313>.
- [5] Belcastro, C. M., and Foster, J. V., "Aircraft loss-of-control accident analysis," *AIAA Guidance, Navigation, and Control Conference*, 2010. <https://doi.org/10.2514/6.2010-8004>, aIAA Paper 2010-8004.

- [6] Federal Aviation Administration, “Airplane Flying Handbook, Chapter 5: Maintaining Aircraft Control: Upset Prevention and Recovery Training,” Tech. Rep. FAA-H-8083-3C, U.S. Department of Transportation, Federal Aviation Administration, Washington, DC, 2021. https://www.faa.gov/regulations_policies/handbooks_manuals/aviation/airplane_handbook.
- [7] UK Civil Aviation Authority, “Stall/Spin Awareness,” Handling Sense Leaflet 02, v2, Civil Aviation Authority, 2009.
- [8] Bellman, R., *Dynamic Programming*, Princeton University Press, Princeton, NJ, 1957.
- [9] Bertsekas, D. P., *Dynamic Programming and Optimal Control*, 4th ed., Vol. 1, Athena Scientific, Belmont, MA, 2017.
- [10] Crespo, L. G., Kenny, S. P., Cox, D. E., and Murri, D. G., “Analysis of control strategies for aircraft flight upset recovery,” *AIAA Guidance, Navigation, and Control Conference*, 2012. <https://doi.org/10.2514/6.2012-5026>, aIAA Paper 2012-5026.
- [11] Cunis, T., Burlion, L., and Condomines, J.-P., “Piecewise polynomial modeling for control and analysis of aircraft dynamics beyond stall,” *Journal of Guidance, Control, and Dynamics*, Vol. 42, No. 4, 2019, pp. 949–957. <https://doi.org/10.2514/1.6003618>.
- [12] Grillo, A., Torre, G. G., and Bunge, R. A., “Optimal Stall Recovery via Deep Reinforcement Learning for a General Aviation Aircraft,” *AIAA SCITECH 2024 Forum*, Orlando, FL, 2024. <https://doi.org/10.2514/6.2024-1280>.
- [13] Munos, R., and Moore, A. W., “Variable Resolution Discretization in Optimal Control,” *Machine Learning*, Vol. 49, No. 2–3, 2002, pp. 291–323. <https://doi.org/10.1023/A:1017992615625>.
- [14] Federal Aviation Administration, “Stall and Spin Awareness Training,” Tech. Rep. Advisory Circular AC 61-67C, U.S. Department of Transportation, Federal Aviation Administration, Washington, DC, 2000. With Changes 1–2.
- [15] Stough, I., H. Paul, DiCarlo, D. J., and Patton, J., James M., “Flight investigation of stall, spin, and recovery characteristics of a low-wing, single-engine, T-tail light airplane,” Tech. Rep. NASA TP-2427, NASA Langley Research Center, 1985.
- [16] Bui, M., Hu, H., He, C., Lu, M., Giovanis, G., Shriraman, A., and Chen, M., “OptimizedDP: an efficient, user-friendly library for optimal control and dynamic programming,” *arXiv preprint arXiv:2204.05520*, 2022.
- [17] Courant, R., Friedrichs, K., and Lewy, H., “On the Partial Difference Equations of Mathematical Physics,” *IBM Journal of Research and Development*, Vol. 11, No. 2, 1967, pp. 215–234. <https://doi.org/10.1147/rd.112.0215>, english translation of the 1928 original in *Mathematische Annalen*.
- [18] Betts, J. T., “Survey of numerical methods for trajectory optimization,” *Journal of Guidance, Control, and Dynamics*, Vol. 21, No. 2, 1998, pp. 193–207. <https://doi.org/10.2514/2.4231>.
- [19] Andersson, J. A. E., Gillis, J., Horn, G., Rawlings, J. B., and Diehl, M., “CasADi: a software framework for nonlinear optimization and optimal control,” *Mathematical Programming Computation*, Vol. 11, No. 1, 2019, pp. 1–36. <https://doi.org/10.1007/s12532-018-0139-4>.
- [20] Wächter, A., and Biegler, L. T., “On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming,” *Mathematical Programming*, Vol. 106, No. 1, 2006, pp. 25–57. <https://doi.org/10.1007/s10107-004-0559-y>.
- [21] Filippov, A. F., *Differential Equations with Discontinuous Righthand Sides*, Mathematics and Its Applications (Soviet Series), Vol. 18, Kluwer Academic Publishers, Dordrecht, 1988. <https://doi.org/10.1007/978-94-015-7793-9>.
- [22] Utkin, V. I., “Variable structure systems with sliding modes,” *IEEE Transactions on Automatic Control*, Vol. AC-22, No. 2, 1977, pp. 212–222. <https://doi.org/10.1109/TAC.1977.1101446>.
- [23] Bryson, A. E., and Ho, Y.-C., *Applied Optimal Control: Optimization, Estimation, and Control*, Hemisphere Publishing, Washington, DC, 1975.